

INVESTIGACIÓN Y ANÁLISIS INTEGRAL DEL STACK TECNOLÓGICO

Django + React

Grupo 8

Yesid Armando Amaya Canarí

Steven Alejandro Araque Castro

Programa: Análisis y Desarrollo de Software

Código: 228118

Instructor: Gustavo Jiménez Suancha

Ficha: 3171062

Fecha: Julio de 2026

PROYECTO FORMATIVO

Construcción de software integrador de tecnologías orientadas a servicios

Fase: Análisis

INTRODUCCIÓN

Como Grupo 8 nos fue asignado el stack Django + React, una combinación que une el backend robusto de Python con Django y Django REST Framework, con un frontend construido en React. A lo largo de esta guía documentamos, con nuestras propias palabras, todo lo que investigamos sobre este ecosistema: su historia, sus componentes, los paradigmas y arquitecturas que lo sustentan, sus patrones de diseño, sus prácticas de seguridad y persistencia, su lugar en el mercado laboral y, finalmente, tres propuestas de proyecto que podríamos desarrollar en las siguientes guías. Cada bloque incluye al final las fuentes que consultamos, para que cualquier persona pueda verificar o profundizar en la información.

BLOQUE 1. IDENTIFICACIÓN DEL STACK

¿Qué es el stack Django + React?

Django + React es un stack tecnológico full-stack que combina un backend construido con Django (framework de Python) y Django REST Framework para exponer una API RESTful, con un frontend construido con React (biblioteca de JavaScript para interfaces de usuario). La comunicación entre ambas capas se realiza a través de peticiones HTTP que consumen endpoints JSON, lo que permite que el frontend y el backend sean completamente independientes: cada uno puede desarrollarse, probarse y desplegarse por separado, y hasta reemplazarse sin afectar al otro.

Django

¿Qué es Django? Django es un framework web de Python de código abierto que facilita la creación de sitios web complejos. Se enfoca en la reutilización de componentes (principio DRY) e incluye funciones listas para usar como sistema de autenticación, conexión a base de datos, panel de administración automático y operaciones CRUD.

Historia y origen. Django nació en 2003 dentro del diario Lawrence Journal-World, en Lawrence, Kansas (Estados Unidos). Sus creadores, Adrian Holovaty y Simon Willison, necesitaban construir aplicaciones web periodísticas en plazos de horas, no de semanas. De esa presión surgió un conjunto de herramientas reutilizables que se convirtió en un framework completo. Fue liberado al público bajo licencia BSD en julio de 2005. Su nombre es un homenaje al guitarrista de jazz gitano Django Reinhardt. La primera versión oficial estable, Django 1.0, se lanzó en septiembre de 2008, e incorporó ya la arquitectura MVT y el ORM que hoy lo caracterizan. En junio de 2008 se creó la Django Software Foundation (DSF), organización sin ánimo de lucro que desde entonces mantiene y protege el proyecto.

Empresa o comunidad responsable. El desarrollo de Django está respaldado por la Django Software Foundation (DSF), una organización sin ánimo de lucro cuyo objetivo es promover, apoyar y avanzar en el proyecto de código abierto.

Evolución y versiones. Django ha mantenido una evolución conservadora, priorizando la seguridad y la estabilidad. Entre 2008 y 2013 las versiones 1.x consolidaron el ORM, el sistema de plantillas y el panel de administración. Entre 2014 y 2017 las versiones 1.7 a 1.11 introdujeron las migraciones automáticas de base de datos (un hito importante) y el nuevo sistema de middlewares. Entre 2018 y 2020 las versiones 2.0 a 3.0 eliminaron el soporte para Python 2 e introdujeron soporte experimental para ASGI. Entre 2021 y 2023 las versiones 3.2 LTS a 4.2 LTS completaron la transición a ASGI nativo. En 2024 llegó Django 5.0 con vistas asíncronas mejoradas, y en 2025 se lanzó Django 5.2 LTS (soporte extendido hasta abril de 2028) junto con Django 6.0, que es la versión más reciente pero con soporte corto (aproximadamente 8 meses), por lo que para proyectos en producción se recomienda usar la rama 5.2 LTS.

Estado actual, comunidad y licenciamiento. Django cuenta con una comunidad global activa que participa en el Django Developer Survey anual organizado junto a JetBrains, foros oficiales y un canal de Discord. Se distribuye bajo licencia BSD de 3 cláusulas, una licencia permisiva que permite uso comercial, modificación y redistribución con muy pocas restricciones.

React

¿Qué es React? React es una biblioteca de JavaScript de código abierto desarrollada por Meta (anteriormente Facebook) para construir interfaces de usuario interactivas. A diferencia de un framework completo, React se centra exclusivamente en la capa de vista, lo que permite al equipo elegir las herramientas complementarias (enrutamiento, manejo de estado, peticiones HTTP) según las necesidades del proyecto.

Historia y origen. React fue creado por Jordan Walke, un ingeniero de Facebook, y se presentó internamente en 2011. Su creación respondió a la necesidad de manejar interfaces con actualizaciones frecuentes y en tiempo real (como el feed de noticias y las notificaciones) sin recargar la página completa. Fue liberado como proyecto de código abierto en mayo de 2013.

Empresa o comunidad responsable. React es mantenido por Meta y por una comunidad global de contribuidores. A diferencia de Django, no existe una fundación independiente; el proyecto se rige por un modelo de gobernanza corporativa con participación comunitaria.

Evolución y versiones. Entre 2013 y 2015 React se consolidó como la biblioteca de interfaces más adoptada, y en 2015 apareció React Native para desarrollo móvil. Entre 2016 y 2018 la versión 15 mejoró el rendimiento y la versión 16 introdujo Fiber, una reescritura interna del motor de renderizado. Entre 2019 y 2021 la versión 16.8 trajo los Hooks, que revolucionaron la forma de

escribir componentes funcionales, y la versión 17 mejoró la renderización incremental. Entre 2022 y 2023 la versión 18 introdujo concurrencia nativa con Concurrent Mode y transiciones automáticas. En 2024-2025 la versión 19 consolidó Server Components y el React Compiler para optimizar automáticamente el rendimiento.

Estado actual, comunidad y licenciamiento. React tiene una de las comunidades más grandes del ecosistema JavaScript, con millones de descargas semanales en npm. Se distribuye bajo licencia MIT, una de las más permisivas, que permite uso comercial, modificación y redistribución sin restricciones.

Tendencias futuras del stack Django + React

Las tendencias apuntan en varias direcciones complementarias. En el backend, Django avanza hacia la consolidación total de su arquitectura asíncrona (ASGI) para aplicaciones en tiempo real, y gracias a su base en Python se perfila como el framework de referencia para integrar modelos de Inteligencia Artificial y Data Science en la web. En el frontend, React evoluciona hacia Server Components (componentes que se renderizan en el servidor sin enviar JavaScript al cliente) y mejoras automáticas de rendimiento mediante el React Compiler. Como combinación, el stack se fortalece porque Django enciende cada vez mejor como proveedor de APIs para frontends desacoplados, y React como consumidor eficiente de esas APIs, especialmente en proyectos que requieren tanto una interfaz moderna como capacidades de análisis de datos o IA en el servidor.

Fuentes consultadas en este bloque

- Documentación oficial de Django, FAQ General — <https://docs.djangoproject.com/es/5.1/faq/general/>
- Wikipedia — Django (framework) — [https://es.wikipedia.org/wiki/Django_\(framework\)](https://es.wikipedia.org/wiki/Django_(framework))
- Wikipedia (EN) — Django Software Foundation — https://en.wikipedia.org/wiki/Django_Software_Foundation
- EcuRed — Django — <https://www.ecured.cu/Django>
- Wikipedia (EN) — Django (web framework) — [https://en.wikipedia.org/wiki/Django_\(web_framework\)](https://en.wikipedia.org/wiki/Django_(web_framework))
- SoloIngeniería — Django Framework en Python: Guía Completa — <https://www.soloingenieria.org/ingenieria-en-sistemas-computacionales/django-framework-en-python/>
- Django REST Framework — Documentación oficial — <https://www.django-rest-framework.org/>
- Django REST Framework — Autenticación — <https://www.django-rest-framework.org/api-guide/authentication/>
- Documentación oficial de Django — Filosofías de diseño — <https://docs.djangoproject.com/en/6.0/misc/design-philosophies/>
- Documentación oficial de Django — Autenticación — <https://docs.djangoproject.com/en/6.0/topics/auth/>
- Documentación oficial de Django — Panel de administración — <https://docs.djangoproject.com/en/6.0/ref/contrib/admin/>
- Documentación oficial de Django — FAQ General (EN) — <https://docs.djangoproject.com/en/6.0/faq/general/>
- Django Software Foundation — Sitio oficial — <https://www.djangoproject.com/foundation/>
- Documentación oficial de Django — Proceso de lanzamiento y versiones — <https://docs.djangoproject.com/en/dev/internals/release-process/>
- Documentación oficial de Django — Migraciones — <https://docs.djangoproject.com/en/6.0/topics/migrations/>
- Documentación oficial de Django — Soporte ASGI — <https://docs.djangoproject.com/en/6.0/topics/async/>
- Django Developer Survey — JetBrains — <https://lp.jetbrains.com/django-developer-survey/>

- *React — Documentación oficial* — <https://react.dev/>
- *React — Referencia de componentes* — <https://react.dev/reference/react>
- *React — Server Components* — <https://react.dev/reference/rsc/server-components>
- *React — Novedades de React 19* — <https://react.dev/blog/2024/12/05/react-19>

BLOQUE 2. COMPONENTES TECNOLÓGICOS

Frontend: React

Lenguaje. JavaScript (y, en proyectos más recientes, TypeScript, aunque no es obligatorio). JavaScript es un lenguaje de programación interpretado, multiparadigma y de tipado dinámico que se ejecuta en el navegador del usuario.

Biblioteca. React. No es un framework completo sino una biblioteca centrada en la capa de vista, lo que significa que el equipo debe elegir las herramientas complementarias (enrutamiento, estado global, peticiones HTTP) según las necesidades del proyecto.

Framework. Este stack no impone un framework de frontend; React se usa directamente como biblioteca de vistas. En proyectos que requieren renderizado en servidor o SEO, se suele combinar con Next.js, pero en una API desacoplada con Django esto no es necesario.

Arquitectura. Basada en componentes reutilizables e independientes. Cada componente encapsula su propia lógica y presentación. El flujo de datos es unidireccional: de componentes padres a hijos mediante props. Se apoya en JSX (sintaxis que permite escribir estructuras tipo HTML dentro de JavaScript) y en el Virtual DOM (representación en memoria de la interfaz que calcula eficientemente qué partes cambian antes de tocar el DOM real).

Características. Uso de Hooks (useState, useEffect, useContext) para manejar estado y ciclo de vida sin clases. Renderizado declarativo: se describe cómo debe verse la interfaz según el estado, y React se encarga de actualizar el DOM. Soporte para React Router (enrutamiento) y Redux, Zustand o Context API (estado global).

Ventajas. Curva de aprendizaje accesible una vez se entiende JSX y el manejo de estado. Ecosistema enorme de librerías y comunidad muy activa. Alto rendimiento gracias al Virtual DOM. Reutilización de conocimientos hacia React Native para desarrollo móvil.

Desventajas. Al ser solo una biblioteca de vistas, cada equipo debe decidir y ensamblar sus propias herramientas de enrutamiento, estado global y peticiones HTTP, lo que puede generar configuraciones distintas entre proyectos. El SEO y el rendimiento en la primera carga pueden ser un reto en una SPA pura sin renderizado en servidor.

Backend: Django + Django REST Framework

Lenguaje. Python. Lenguaje de programación interpretado, multiparadigma (orientado a objetos, funcional, imperativo), de tipado dinámico y sintaxis clara y legible.

Framework. Django. Framework web de alto nivel que sigue la arquitectura MVT (Modelo-Vista-Template), una variante de MVC donde el propio framework asume el rol de controlador.

Runtime. CPython, el intérprete oficial de Python. Django corre sobre servidores WSGI como Gunicorn para vistas síncronas, o ASGI (Uvicorn/Daphne) cuando se usan vistas asíncronas.

Arquitectura. Django REST Framework (DRF) complementa a Django exponiendo la lógica de negocio como una API RESTful que React consume mediante endpoints JSON. DRF añade serializadores, vistas basadas en clases (ViewSet), un sistema de permisos y autenticación (Token, JWT, Session) y un explorador de API navegable para probar endpoints directamente desde el navegador.

Características. Incluye "baterías incluidas": ORM, sistema de autenticación, panel de administración automático, manejo de formularios y protección de seguridad básica de fábrica. Sistema de migraciones para versionar los cambios del modelo de datos sin perder información.

Ventajas. Muy productivo para levantar APIs completas en poco tiempo gracias a su ORM y su generación automática de formularios y panel administrativo. Seguridad reforzada por defecto (protección CSRF, sanitización de consultas SQL, cabeceras seguras). Comunidad y documentación oficial extensas y en varios idiomas.

Desventajas. Puede sentirse "pesado" para microservicios muy pequeños, comparado con opciones más minimalistas como Flask o FastAPI. El manejo nativo de operaciones asíncronas es más reciente que en frameworks diseñados desde el inicio para ese propósito.

Base de datos

Tipo. Relacional (SQL). Los datos se organizan en tablas con filas y columnas, y las relaciones entre tablas se definen explícitamente mediante llaves foráneas.

Motor. PostgreSQL o MySQL, dos motores relacionales maduros y ampliamente soportados por el ORM de Django. PostgreSQL destaca por su soporte avanzado de tipos de datos (JSON, arrays, texto completo) y por su cumplimiento estricto del estándar SQL. MySQL es reconocido por su simplicidad y enorme adopción en aplicaciones web tradicionales.

Modelo. Relacional con relaciones uno a uno, uno a muchos (ForeignKey) y muchos a muchos (ManyToManyField), representadas en Django mediante sus correspondientes campos.

Características. Integridad referencial garantizada por el motor, transacciones ACID y soporte maduro de índices. PostgreSQL añade ventajas como campos JSON nativos, full-text search y tipos avanzados (ArrayField, RangeField).

Ventajas. Consistencia y seguridad de los datos gracias al modelo relacional. Consultas predecibles y optimizables. Ecosistema maduro de herramientas de respaldo, replicación y monitoreo.

Desventajas. Escalar horizontalmente una base de datos relacional es más complejo que con una base NoSQL, aunque para la mayoría de aplicaciones empresariales esto no es un problema real.

Herramientas adicionales del stack

ORM: Django ORM (qué es y para qué sirve)

Es el componente que traduce las clases de Python (modelos) en tablas y consultas SQL, permitiendo al desarrollador interactuar con la base de datos usando sintaxis de Python en lugar de escribir SQL manualmente.

Cómo funciona. Django inspecciona las clases que heredan de `models.Model`, genera automáticamente las sentencias SQL correspondientes (CREATE TABLE, SELECT, INSERT, UPDATE, DELETE) según las operaciones que el desarrollador realiza en Python, y devuelve los resultados como instancias de los modelos.

Características. Soporta relaciones uno a uno, uno a muchos y muchos a muchos. Permite encadenar consultas mediante QuerySets (`.filter()`, `.exclude()`, `.order_by()`). Genera migraciones automáticas al detectar cambios en los modelos. Ofrece un sistema de expresiones F y Q para consultas complejas.

Arquitectura. Actúa como una capa de abstracción entre el código Python y la base de datos: el desarrollador escribe código Python, el ORM lo traduce a SQL optimizado para el motor correspondiente (PostgreSQL o MySQL), y los resultados los convierte de vuelta en objetos Python. Esto permite cambiar de motor de base de datos sin modificar las consultas.

Ventajas. Productividad alta al evitar escribir SQL manual. Seguridad automática contra inyección SQL (parametriza las consultas). Portabilidad entre motores de base de datos.

Desventajas. En consultas muy complejas el ORM puede generar SQL ineficiente, y a veces es necesario recurrir a SQL crudo con `.raw()` o `.execute()`. No expone todas las características avanzadas de cada motor de base de datos.

Casos de uso. Todas las operaciones de base de datos del proyecto: crear registros, consultar con filtros, actualizar datos y eliminar registros. Ejemplo: `Producto.objects.filter(precio__gt=100)` en lugar de `SELECT * FROM producto WHERE precio > 100`.

Alternativas. SQLAlchemy (más flexible y explícito, pero más verboso), peewee (más ligero y simple), Tortoise ORM (orientado a operaciones asíncronas).

Autenticación: JWT (JSON Web Token) (qué es y para qué sirve)

Es un estándar abierto que representa de forma compacta y verificable la identidad de un usuario autenticado mediante un token firmado digitalmente. En una arquitectura desacoplada como Django + React, el backend emite el token y el frontend lo envía en cada petición para demostrar que el usuario está autenticado sin que el servidor mantenga estado de sesión.

Cómo funciona. El usuario envía sus credenciales (usuario y contraseña) a un endpoint de login. Django las valida, genera un token JWT firmado con la clave secreta del servidor (SECRET_KEY) y lo devuelve al frontend. El frontend almacena el token y lo incluye en el header Authorization: Bearer de cada petición posterior. El backend verifica la firma del token en cada request sin necesidad de consultar la base de datos.

Características. El token tiene tres partes: header (algoritmo de firma), payload (datos del usuario, como ID y roles) y signature (firma digital que garantiza integridad). Soporta tiempos de expiración (exp claim) y tokens de refresco (refresh tokens) para renovar la sesión sin que el usuario vuelva a loguearse.

Arquitectura. El flujo sigue el modelo stateless: el servidor no guarda ninguna información de sesión. Toda la información necesaria viaja dentro del token. Esto permite escalar el backend horizontalmente (múltiples servidores) sin necesidad de compartir estado de sesión entre ellos.

Ventajas. Liviano y escalable al no requerir estado en el servidor. Ideal para arquitecturas desacopladas donde frontend y backend están en dominios distintos. Funciona bien con aplicaciones móviles que también consumen la API.

Desventajas. Exige buenas prácticas de expiración (tokens de corta duración) y almacenamiento seguro en el navegador (no es recomendable guardarlos en localStorage sin protección adicional). Una vez emitido, un token no puede revocarse fácilmente antes de que expire a menos que se implemente una lista negra.

Casos de uso. Autenticación en APIs REST consumidas por React. Login, logout, rutas protegidas en el frontend y permisos por rol en el backend.

Alternativas. Tokens de sesión de Django (para aplicaciones monolíticas donde el servidor renderiza el HTML), OAuth 2.0 (para login con terceros como Google o GitHub), SimpleJWT con djangorestframework-simplejwt (la implementación más usada en Django).

Gestión de dependencias: pip / Poetry y npm / yarn (qué es y para qué sirve)

Son las herramientas que gestionan las librerías de terceros que necesita el proyecto: pip o Poetry para el backend en Python, y npm o yarn para el frontend en JavaScript. Su función es listar, instalar, actualizar y resolver las dependencias de cada proyecto de forma reproducible.

Cómo funciona. En el backend, pip lee el archivo requirements.txt (o Poetry lee pyproject.toml), descarga las librerías desde PyPI (el repositorio oficial de Python) y las instala junto con sus dependencias transitivas. En el frontend, npm o yarn leen package.json, descargan los paquetes desde el registro npm y los instalan en la carpeta node_modules.

Características. pip es el gestor oficial de Python, simple pero sin bloqueo de versiones. Poetry añade bloqueo de versiones (poetry.lock), gestión de entornos virtuales integrada y resolución de dependencias más estricta. npm es el gestor oficial de Node.js. yarn ofrece instalación más rápida en algunos casos y también genera un archivo de bloqueo (yarn.lock).

Arquitectura. Cada proyecto tiene su propio archivo de dependencias aislado. El backend (carpeta del proyecto Django) y el frontend (carpeta de React) son proyectos independientes con sus propios gestores, sus propios archivos de configuración y sus propias carpetas de instalación, aunque pueden convivir en un mismo repositorio (monorepo) o en repositorios separados.

Ventajas. Reproducibilidad: cualquier desarrollador puede recrear el mismo entorno ejecutando pip install o npm install. Control de versiones de las dependencias para evitar que una actualización rompa el proyecto.

Desventajas. Conflictos de versiones entre dependencias que requieren versiones incompatibles de una misma librería. La carpeta node_modules puede ser muy grande y no debe subirse al repositorio.

Casos de uso. Configurar el entorno de desarrollo en una máquina nueva. Desplegar el proyecto en un servidor asegurando que se instalen exactamente las mismas versiones.

Alternativas. Pipenv o Conda (Python), pnpm o Bun (JavaScript, más rápidos que npm).

Control de versiones: Git y GitHub/GitLab (qué es y para qué sirve)

Git es un sistema de control de versiones distribuido que registra el historial completo de cambios del código fuente. GitHub y GitLab son plataformas en la nube que alojan repositorios Git y agregan funcionalidades colaborativas como control de ramas, revisión de código y automatización de despliegues.

Cómo funciona. Git crea instantáneas (commits) del estado del proyecto en cada punto de guardado. Las ramas (branches) permiten trabajar en funcionalidades aisladas sin afectar la rama principal. Las pull requests (o merge requests) permiten que otro integrante revise los cambios antes de integrarlos a la rama principal.

Características. Git es distribuido: cada desarrollador tiene una copia completa del repositorio con todo su historial. Soporta fusión (merge) y rebase para integrar cambios. GitHub/GitLab añaden interfaces web, integración continua (CI/CD), gestión de issues y wikis de documentación.

Arquitectura. El backend (Django) y el frontend (React) pueden versionarse en un mismo repositorio (monorepo, con carpetas separadas) o en repositorios independientes (uno para el backend, otro para el frontend). En un monorepo, un solo pull request puede incluir cambios en ambos lados, lo que facilita la trazabilidad pero requiere estrategias de CI/CD más complejas.

Ventajas. Trazabilidad completa: se sabe quién cambió qué, cuándo y por qué. Trabajo en paralelo sin pisar el código de otros integrantes. Posibilidad de revertir cambios si algo sale mal.

Desventajas. Curva de aprendizaje inicial para entender ramas, fusiones y resolución de conflictos. En equipos grandes, los conflictos de fusión pueden ser frecuentes si no se coordinan bien las ramas.

Casos de uso. Todo proyecto profesional de software: registrar cambios, coordinar el trabajo en equipo, revisar código antes de integrarlo y desplegar automáticamente desde la rama principal.

Alternativas. Mercurial (otro sistema distribuido, menos popular), SVN (centralizado, en desuso para nuevos proyectos). Como plataformas: Bitbucket (de Atlassian), Gitea (self-hosted).

Testing: pytest-django y Jest + React Testing Library (qué es y para qué sirve)

Son las herramientas que permiten escribir y ejecutar pruebas automatizadas para verificar que el código funciona correctamente. En el backend, pytest-django extiende pytest para probar vistas, modelos y serializadores de Django. En el frontend, Jest ejecuta las pruebas y React Testing Library simula la interacción del usuario con los componentes.

Cómo funciona. Se escriben funciones de prueba que ejecutan una parte del código y verifican que el resultado es el esperado (por ejemplo, que un serializador rechace un dato inválido, o que un componente renderice un botón al recibir ciertas props). El framework de pruebas ejecuta todas las funciones, reporta cuáles pasaron y cuáles fallaron, y muestra el error exacto de las que fallaron.

Características. pytest-django ofrece sintaxis más limpia que unittest de Python, fixtures reutilizables y cobertura de código. Jest ofrece ejecución en paralelo (rápido), mocking automático de módulos y snapshots (guardar una foto del componente y comparar en futuras ejecuciones). React Testing Library se enfoca en probar el comportamiento desde la perspectiva del usuario (hacer clic, escribir texto) en lugar de probar detalles de implementación interna.

Arquitectura. Las pruebas se organizan en tres niveles: unitarias (prueban una función o método aislado), de integración (prueban que varios componentes trabajan juntos, como una vista que guarda en la base de datos) y E2E (simulan el flujo completo del usuario desde el navegador con herramientas como Cypress o Playwright, aunque estas no se incluyen en esta categoría específica).

Ventajas. Detectar errores antes de que lleguen a producción. Permiten refactoring seguro: se cambia el código y se ejecutan las pruebas para verificar que nada se rompió. Documentan el comportamiento esperado del sistema.

Desventajas. Requieren tiempo de escritura y mantenimiento (si el código cambia, las pruebas deben actualizarse). En proyectos con fechas ajustadas pueden percibirse como una carga, aunque a largo plazo ahorran más tiempo del que invierten.

Casos de uso. Verificar que un serializador de DRF valide correctamente los datos de entrada. Probar que un componente de React renderice la información de un usuario cuando recibe las props adecuadas. Verificar que una vista de Django devuelva el código HTTP esperado (200, 404, 201).

Alternativas. unittest (incluso en Django, sin pytest), nose2 (Python). En el frontend: Vitest (más rápido que Jest), Enzyme (enfocado en detalles internos, en desuso). Para E2E: Cypress o Playwright.

IDE y herramientas de desarrollo: Visual Studio Code y PyCharm (qué es y para qué sirve)

Son los entornos de desarrollo donde se escribe, depura y ejecuta el código. VS Code es un editor de código fuente ligero y extensible. PyCharm es un IDE especializado en Python con funcionalidades integradas para Django.

Cómo funciona. Ambos ofrecen resaltado de sintaxis, autocompletado inteligente, depurador paso a paso, terminal integrada y extensión para control de versiones con Git. VS Code logra estas capacidades mediante extensiones (Python, Prettier, ESLint, React Native Tools). PyCharm las trae integradas de fábrica, especialmente las relacionadas con Django (navegación de modelos, soporte de plantillas, panel de base de datos).

Características. VS Code es gratuito, ligero (consume pocos recursos), muy extensible y soporta múltiples lenguajes con la misma instalación. PyCharm existe en versión Community (gratuita) y Professional (de pago, con soporte para Django, bases de datos y herramientas de desarrollo web avanzadas).

Arquitectura. Ambos se conectan al intérprete de Python del entorno virtual del proyecto para ofrecer autocompletado y depuración específicos del contexto del código. También se conectan al servidor de desarrollo de Django (runserver) y al de React (npm run dev) para ejecutar y depurar la aplicación sin salir del editor.

Ventajas. Autocompletado que acelera la escritura de código. Depurador que permite detener la ejecución en puntos específicos para inspeccionar variables. Integración con Git para ver cambios y resolver conflictos sin salir del editor.

Desventajas. VS Code requiere instalar y configurar extensiones manualmente, lo que puede tomar tiempo al inicio. PyCharm Professional es de pago y consume más recursos de memoria, especialmente en proyectos grandes.

Casos de uso. Escritura y depuración del código del backend (Django) y del frontend (React) en un mismo entorno. Navegación rápida entre archivos, definiciones y referencias.

Alternativas. WebStorm (de pago, especializado en JavaScript/TypeScript), Sublime Text (muy ligero pero con menos funcionalidades integradas), Neovim (para usuarios avanzados de terminal).

Celery (qué es y para qué sirve)

Es una cola de tareas distribuida que permite ejecutar procesos pesados o programados fuera del ciclo petición-respuesta HTTP, apoyándose en un intermediario de mensajes (broker) como Redis o RabbitMQ para recibir y distribuir las tareas.

Cómo funciona. El backend de Django envía una tarea a la cola (por ejemplo, "enviar correo de bienvenida") en lugar de ejecutarla directamente. Un worker de Celery (proceso independiente que corre en paralelo) toma la tarea de la cola, la ejecuta en segundo plano y guarda el resultado. El usuario recibe la respuesta HTTP inmediatamente sin esperar a que la tarea termine.

Características. Soporta ejecución asíncrona, programación de tareas recurrentes (cron-like con celery-beat), encadenamiento de tareas (una tarea dispara otra cuando termina), grupos de tareas (ejecutar varias en paralelo y esperar a que todas terminen) y reintento automático de tareas fallidas.

Arquitectura. Celery sigue una arquitectura de productor-consumidor: Django es el productor (envía tareas a la cola), el broker (Redis o RabbitMQ) almacena las tareas pendientes, y uno o más workers son los consumidores (toman tareas y las ejecutan). Los workers pueden correr en la misma máquina que Django o en máquinas separadas, lo que permite escalar el procesamiento de tareas de forma independiente al servidor web.

Ventajas. No bloquea las respuestas al usuario: tareas largas se ejecutan en segundo plano. Permite programar tareas recurrentes (enviar reportes cada lunes, limpiar datos cada noche). Los workers pueden escalarse horizontalmente (más máquinas) según la carga de trabajo.

Desventajas. Complejidad adicional en la infraestructura: se necesita instalar y mantener un broker (Redis o RabbitMQ) además de los workers. El monitoreo y la depuración de tareas en segundo plano es más difícil que la de código síncrono.

Casos de uso. Envío de correos transaccionales o notificaciones. Generación de reportes pesados (PDFs con muchos datos). Procesamiento de imágenes o archivos. Llamadas a APIs externas que tardan varios segundos. En el contexto de VocalCode: procesar la interpretación de voz con IA sin que el usuario espere.

Alternativas. Huey (más simple, usa Redis o base de datos como broker), RQ (más ligero, solo Redis), Django Q (para tareas asíncronas simples sin broker externo), Background Tasks de FastAPI (si se migrara a FastAPI).

Servicios externos (qué son y para qué sirven)

Son servicios de terceros que se integran al proyecto para añadir funcionalidades sin tener que desarrollarlas desde cero. Los más comunes en este stack son: Redis (caché y broker de Celery), servicios de correo transaccional (SendGrid, Mailgun), pasarelas de pago (Stripe, MercadoPago) y almacenamiento en la nube (AWS S3) para archivos estáticos y multimedia.

Cómo funciona. Cada servicio expone su propia API que el backend de Django consume mediante peticiones HTTP o librerías cliente específicas. Por ejemplo, para enviar un correo con SendGrid, Django usa la librería `sendgrid-python` que envía los datos del correo a los servidores de SendGrid; para almacenar un archivo en S3, Django usa `django-storages` que gestiona la conexión con AWS.

Características. Redis es una base de datos en memoria, extremadamente rápida, usada como caché (almacenar resultados de consultas frecuentes) y como broker de mensajes para Celery. Los servicios de correo ofrecen entregabilidad alta (evitar que los correos lleguen a spam), plantillas, estadísticas de apertura y clics. Las pasarelas de pago manejan la complejidad de procesar tarjetas, suscripciones y reembolsos cumpliendo normativas de seguridad (PCI DSS).

Arquitectura. Los servicios externos se integran en la capa de servicios del backend, nunca directamente en las vistas o en el frontend. Esto permite abstraer la lógica de comunicación con el servicio externo en una clase o función específica, de modo que si se cambia de proveedor (por ejemplo, de SendGrid a Mailgun) solo se modifica esa capa sin afectar el resto del código.

Ventajas. Funcionalidades maduras y confiables sin tener que desarrollarlas. Escalabilidad delegada al proveedor (no hay que escalar la infraestructura de correo o pagos). Actualizaciones y mantenimiento a cargo del proveedor.

Desventajas. Dependencia de servicios de terceros: si el servicio se cae, la funcionalidad se cae con él. Costos por uso que pueden crecer con el tráfico. Latencia adicional por las llamadas a servicios externos.

Casos de uso. Caché de consultas frecuentes a la base de datos con Redis. Envío de correos de verificación, recuperación de contraseña y notificaciones. Pagos en línea con Stripe o MercadoPago. Almacenamiento de imágenes de perfil y archivos subidos por usuarios en AWS S3.

Alternativas. Para caché y broker: Memcached (alternativa a Redis, sin persistencia). Para correo: SMTP directo con Gmail (sin costo pero menos confiable). Para almacenamiento: MinIO (compatible con S3, self-hosted, sin costo de uso). Para pagos: PayPal, MercadoPago según el mercado objetivo.

Fuentes consultadas en este bloque

- Wikipedia — Django (framework), arquitectura MVT — [https://es.wikipedia.org/wiki/Django_\(framework\)](https://es.wikipedia.org/wiki/Django_(framework))
- Django ORM — Consultas y QuerySets — <https://docs.djangoproject.com/en/6.0/topics/db/queries/>
- Django REST Framework — Documentación oficial — <https://www.django-rest-framework.org/>
- JWT.io — Introduction to JSON Web Tokens — <https://jwt.io/introduction>
- Estándar Oficial JWT (RFC 7519) — <https://datatracker.ietf.org/doc/html/rfc7519>
- SimpleJWT — Implementación JWT para Django — <https://django-rest-framework-simplejwt.readthedocs.io/en/latest/>
- Poetry — Documentación oficial — <https://python-poetry.org/docs/>
- npm — Documentación oficial — <https://docs.npmjs.com/>
- Yarn — Documentación oficial — <https://yarnpkg.com/getting-started>
- Git — Pro Git (Libro oficial en español) — <https://git-scm.com/book/es/v2>
- pytest-django — Documentación oficial — <https://pytest-django.readthedocs.io/en/latest/>
- Jest — Documentación oficial — <https://jestjs.io/docs/getting-started>
- React Testing Library — Documentación oficial — <https://testing-library.com/docs/react-testing-library/intro/>
- Celery — Documentación oficial — <https://docs.celeryq.dev/en/stable/>
- Redis — Documentación oficial — <https://redis.io/docs/>
- Visual Studio Code — Documentación oficial — <https://code.visualstudio.com/docs>
- PyCharm — Documentación oficial — <https://www.jetbrains.com/pycharm/documentation/>
- React — Documentación oficial — <https://react.dev/>
- PostgreSQL — About — <https://www.postgresql.org/about/>

BLOQUE 3. PARADIGMAS DE PROGRAMACIÓN

Un paradigma de programación es un estilo o enfoque fundamental para estructurar y organizar el código. Ni Python ni JavaScript pertenecen a un solo paradigma: ambos son lenguajes multiparadigma, lo que permite combinar distintos estilos dentro de un mismo proyecto según las necesidades de cada capa.

Paradigmas presentes en el stack

Programación imperativa. Describe paso a paso cómo hacer algo mediante instrucciones secuenciales, bucles y condicionales que modifican el estado del programa. Es el estilo más tradicional y constituye la base de ambos lenguajes, aunque no es el enfoque dominante en ninguna de las capas del stack.

Programación declarativa. Describe qué se quiere obtener, no cómo obtenerlo. El desarrollador declara el resultado deseado y el sistema se encarga de los pasos internos para lograrlo. Es el estilo dominante en React (al describir la interfaz según el estado) y en el ORM de Django (al declarar qué datos se quieren consultar sin escribir SQL).

Programación orientada a objetos. Organiza el código en objetos que combinan datos y comportamiento en clases que sirven como plantillas. Es el paradigma dominante en el backend de Django: los modelos, las vistas basadas en clases y los serializadores de DRF se definen como clases que heredan comportamiento de clases base del framework.

Programación funcional. Trata la computación como evaluación de funciones matemáticas: evita el cambio de estado, prefiere funciones puras y promueve la composición de funciones pequeñas. Es el estilo dominante en React a través de los componentes funcionales y los Hooks, que promueven inmutabilidad del estado y funciones puras. En Django aparece de forma puntual en funciones lambda, comprensiones de listas y funciones de orden superior de Python.

Programación reactiva. El flujo del programa se guía por cambios en un flujo de datos: cuando un valor cambia, las partes que dependen de él se actualizan automáticamente. React tiene elementos reactivos (useEffect responde a cambios de estado) pero no es un framework reactivo puro como lo sería RxJS. En el backend de Django este paradigma no se aplica de forma directa.

Programación basada en eventos. El flujo del programa se guía por la ocurrencia de eventos: el sistema escucha acciones y ejecuta funciones específicas cuando ocurren. En el frontend, toda interacción del usuario (clics, envíos de formularios) se maneja mediante event handlers. En el backend, Django implementa este paradigma a través de sus signals (señales), que permiten que distintas partes de la aplicación reaccionen cuando ocurre un evento sin acoplar directamente el código que dispara el evento con el que reacciona.

Programación asíncrona. Permite que el programa continúe ejecutando otras tareas mientras espera por una operación lenta. En el frontend, las peticiones a la API de Django se resuelven con promesas y async/await. En el backend, Django soporta vistas asíncronas desde la versión 3.1 usando async def, lo que permite atender otras peticiones mientras espera por operaciones de entrada/salida.

Programación concurrente. Permite que múltiples tareas avancen al mismo tiempo. En el backend de Django, la concurrencia se maneja principalmente a través de Celery (tareas que se ejecutan en procesos paralelos fuera del hilo principal del servidor web) y del servidor ASGI (Uvicorn o Daphne) para manejar múltiples conexiones simultáneas en vistas asíncronas.

Paradigmas predominantes por capa

En el frontend (React) predominan la programación declarativa y la programación funcional: los componentes son funciones que reciben datos y devuelven una descripción de la interfaz, sin manipular el DOM directamente. En el backend (Django) predomina la programación orientada a objetos: modelos, vistas y serializadores se definen como clases que heredan de clases base del framework. En la base de datos, a través del ORM de Django, predomina la programación declarativa: el desarrollador describe qué datos quiere obtener mediante QuerySets y Django traduce esa declaración a SQL. La programación asíncrona y la basada en eventos son transversales: están presentes en ambas capas pero no son el estilo dominante en ninguna de ellas.

Aplicación por capa

Frontend (React). Los componentes se definen como funciones puras que reciben datos mediante props y devuelven una descripción de la interfaz según el estado actual (declarativa y funcional). El estado se maneja de forma inmutable: en lugar de modificarlo directamente, se crea una nueva copia con el cambio usando Hooks como useState. Toda interacción del usuario se captura mediante event handlers (onClick, onChange, onSubmit) que ejecutan lógica en respuesta a eventos (basada en eventos). Las peticiones a la API de Django se resuelven con fetch o axios combinados con async/await, lo que permite que la interfaz siga respondiendo mientras espera la respuesta del servidor (asíncrona).

Backend (Django). Los modelos heredan de models.Model, las vistas basadas en clases heredan de ListView, CreateView o ViewSet, y los serializadores heredan de serializers.Serializer, todo lo cual organiza el backend bajo programación orientada a objetos. La lógica interna de los métodos usa condicionales, bucles y asignaciones de variables cuando es necesario (imperativa). Las consultas a la base de datos se expresan como QuerySets declarativos que Django traduce a SQL. Las signals permiten que distintas partes del sistema reaccionen a eventos como la creación de un usuario sin acoplar el código emisor del receptor (basada en eventos). Las vistas definidas con async def permiten manejar múltiples peticiones concurrentes sin bloquear el hilo principal (asíncrona y concurrente). Celery ejecuta tareas pesadas en procesos paralelos independientes del servidor web (concurrente). Funciones lambda, comprensiones de listas y funciones de orden superior aparecen de forma puntual (funcional).

Base de datos (a través del ORM de Django). El desarrollador describe qué datos quiere obtener usando sintaxis de Python (`Producto.objects.filter(precio__gt=100, categoria__nombre="electrónica")`) y Django traduce esa declaración a una sentencia SQL que el motor ejecuta (declarativa). Los modelos de Django representan tablas como clases y las filas como instancias, lo que permite manipular registros como objetos Python (orientada a objetos). Cuando el

ORM no puede generar una consulta eficiente, el desarrollador puede recurrir a SQL crudo usando `Model.objects.raw()` o `connection.cursor().execute()`, lo que implica escribir instrucciones paso a paso (imperativa).

Fuentes consultadas en este bloque

- *React — Describing the UI (declarativa)* — <https://react.dev/learn/describing-the-ui>
- *React — Responding to Events (basada en eventos)* — <https://react.dev/learn/responding-to-events>
- *Django — Class-Based Views (orientación a objetos)* — <https://docs.djangoproject.com/en/stable/topics/class-based-views/>
- *Django — Signals (basada en eventos)* — <https://docs.djangoproject.com/en/stable/topics/signals/>
- *Django — Asynchronous Support (asíncrona)* — <https://docs.djangoproject.com/en/stable/topics/async/>
- *Python — Functional Programming HOWTO* — <https://docs.python.org/3/howto/functional.html>
- *Celery — Documentación oficial* — <https://docs.celeryq.dev/en/stable/>
- *Redux — Programación reactiva (definición de referencia)* — <https://reactivex.io/>

BLOQUE 4. ARQUITECTURA DE SOFTWARE

La arquitectura de software define la estructura general del sistema: cómo se organizan los componentes, cómo se comunican entre sí y qué principios estructurales los rigen. En un stack desacoplado como Django + React, donde el frontend y el backend son independientes y se comunican por API, varias arquitecturas coexisten en distintos niveles de abstracción.

Arquitecturas investigadas

Cliente-Servidor. El sistema se divide en dos partes: un cliente (React, que corre en el navegador del usuario) que envía peticiones, y un servidor (Django, que corre en un servidor remoto) que las recibe, procesa y devuelve una respuesta. Es la base de cualquier aplicación web y el patrón fundamental sobre el cual se construye todo el stack.

MVC (Modelo-Vista-Controlador). Separa la aplicación en tres componentes: el Modelo (datos y lógica de negocio), la Vista (lo que ve el usuario) y el Controlador (recibe las peticiones, coordina Modelo y Vista, y devuelve una respuesta). Django implementa una variante llamada MVT (Modelo-Vista-Template), donde el propio framework asume el rol de controlador y el "template" es la capa de presentación que en este stack prácticamente no se usa porque React reemplaza esa función.

Arquitectura en capas. Organiza el código en capas horizontales con responsabilidades diferenciadas: capa de presentación (vistas, serializadores), capa de aplicación (lógica de coordinación), capa de dominio (lógica de negocio pura) y capa de datos (acceso a la base de datos). Cada capa solo puede depender de la capa inmediatamente inferior, nunca de capas superiores, lo que facilita el mantenimiento y las pruebas.

REST (Representational State Transfer). Es un estilo de arquitectura para diseñar APIs web basado en recursos identificados por URLs y en los métodos HTTP estándar: GET para consultar, POST para crear, PUT/PATCH para actualizar y DELETE para eliminar. Django REST Framework expone la lógica de negocio de Django como una API RESTful que React consume enviando y recibiendo datos en formato JSON a través de estos endpoints.

Microservicios. Divide la aplicación en servicios pequeños e independientes, cada uno responsable de una parte del negocio (por ejemplo, un servicio de usuarios, otro de pagos, otro de notificaciones), que se comunican entre sí mediante APIs o mensajes. Cada microservicio puede desarrollarse, desplegarse y escalarse de forma independiente. Este estilo se reserva para aplicaciones de muy gran escala; en proyectos formativos como este es más adecuado un backend monolítico bien organizado.

Serverless. El desarrollador no administra ningún servidor: el proveedor de nube ejecuta el código solo cuando se necesita (por función invoked) y cobra por ejecución, no por tiempo alquilado. Se adapta bien al frontend estático de React (en plataformas como Vercel o Netlify) pero no al backend de Django, que mantiene conexiones persistentes a la base de datos y no encaja de forma natural en un modelo serverless.

Event-Driven (basada en eventos). El flujo del sistema se guía por la producción y consumo de eventos: cuando ocurre una acción (un usuario se registra, se crea un pedido, se sube un archivo), se emite un evento que uno o varios componentes escuchan y procesan de forma asíncrona. En este stack se manifiesta a través de Celery con un broker de mensajes (Redis o RabbitMQ): ciertas acciones del backend disparan tareas que se procesan en segundo plano sin bloquear la respuesta al usuario.

Clean Architecture. Propuesta de Robert C. Martin que busca que la lógica de negocio (reglas del negocio, entidades) no dependa directamente del framework, la base de datos, la interfaz gráfica ni ningún detalle técnico externo. La lógica de negocio vive en el centro del sistema, rodeada por capas de adaptadores que la conectan con el mundo exterior. Esto permite cambiar de framework, de base de datos o de interfaz sin reescribir el núcleo del sistema.

Arquitectura Hexagonal. Esencialmente el mismo concepto que Clean Architecture pero con una metáfora visual: la lógica de negocio está en el centro (el hexágono) y los puertos (interfaces) definen cómo se comunica con el exterior. Los adaptadores (implementaciones concretas) se conectan a esos puertos desde fuera del hexágono. Un puerto podría ser RepositorioDeProyectos con métodos como guardar() y buscar_por_id(), y el adaptador sería la implementación que usa el ORM de Django para hablar con PostgreSQL.

Arquitecturas más adecuadas para el stack Django + React

Dado que el stack es desacoplado por naturaleza (frontend y backend independientes que se comunican por API), varias arquitecturas se combinan de forma natural en distintos niveles:

- Cliente-Servidor: es la base obligatoria del stack. React es el cliente, Django es el servidor, y la comunicación se da a través de HTTP.
- REST: es el estilo de comunicación entre ambas capas. Django REST Framework expone los recursos como endpoints HTTP que React consume.
- Arquitectura en capas: organiza el código del backend Django en niveles con responsabilidades claras, evitando que toda la lógica quede amontonada en las vistas.
- Clean Architecture / Arquitectura Hexagonal: se aplica en el núcleo del backend para que la lógica de negocio más crítica no dependa de Django, del ORM ni de ningún detalle técnico. Esto permite probar esa lógica de forma aislada y cambiar de tecnología en el futuro sin reescribir el negocio.
- Event-Driven: se aplica de forma complementaria a través de Celery para tareas que no deben bloquear la respuesta al usuario (procesamiento pesado, envío de correos, llamadas a APIs externas).

MVC/MVT está presente internamente en Django pero no es la arquitectura que define el sistema como un todo, porque el "template" de Django no se usa y React asume el rol de vista. Microservicios y Serverless no son adecuados para el alcance de un proyecto formativo: los microservicios añaden complejidad operativa (orquestación, consistencia distribuida) que no se justifica con tres desarrolladores, y Serverless no encaja con la naturaleza de Django como framework con estado persistente de conexión a base de datos.

Justificación técnica

La elección de Cliente-Servidor + REST como base es obligatoria dado que React corre en el navegador y Django en el servidor: no existe alternativa razonable para este stack. La Arquitectura en Capas se justifica porque un backend Django sin capas tiende a acumular toda la lógica en las vistas, lo que hace el código difícil de probar, de mantener y de escalar. Separar en presentación (vistas/serializadores), aplicación (servicios) y datos (repositorios/ORM) permite que cada capa evolucione de forma independiente.

Clean Architecture / Hexagonal se justifica en la lógica de negocio más crítica (por ejemplo, el motor de interpretación de voz de VocalCode) porque esa lógica es el verdadero valor del proyecto y no debería estar acoplada a Django. Si en el futuro se cambia de framework, de base de datos o de proveedor de IA, el núcleo del negocio debe sobrevivir sin modificaciones. El costo de esta separación es un poco más de código de adaptadores iniciales, pero el beneficio es un sistema más robusto y más fácil de probar unitariamente.

Event-Driven mediante Celery se justifica porque hay operaciones que no deben bloquear la respuesta HTTP: si el usuario dicta una instrucción de voz que requiere procesamiento con IA, el backend debe responder inmediatamente ("instrucción recibida, procesando...") y la tarea pesada se ejecuta en segundo plano. Sin este patrón, el usuario esperaría varios segundos mirando una pantalla congelada.

Se descarta MVC puro porque con React como frontend independiente, Django ya no renderiza templates ni maneja estado de sesión, rompiendo el ciclo tradicional Modelo-Vista-Controlador. Se descarta Microservicios porque la complejidad operativa (orquestración, consistencia distribuida, monitoreo multi-servicio) no se justifica en un MVP académico. Se descarta Serverless porque Django no es nativo en FaaS (Function as a Service); forzarlo con herramientas como Zappa introduce cold starts y limitaciones de tamaño que degradan la experiencia del usuario.

Fuentes consultadas en este bloque

- Django — FAQ: ¿Es Django un framework MVC? — <https://docs.djangoproject.com/en/6.0/faq/>
- Robert C. Martin (Uncle Bob) — The Clean Architecture — <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- Alistair Cockburn — Hexagonal Architecture — <https://alistair.cockburn.us/hexagonal-architecture/>
- Martin Fowler — Microservices — <https://martinfowler.com/articles/microservices.html>
- Microsoft Architecture Center — Diseño de API RESTful — <https://learn.microsoft.com/es-es/azure/architecture/best-practices/api-design>
- Django REST Framework — Documentación oficial — <https://www.django-rest-framework.org/>
- Celery — Documentación oficial — <https://docs.celeryq.dev/en/stable/>

BLOQUE 5. PATRONES DE DISEÑO

Los patrones de diseño son soluciones probadas y reutilizables a problemas recurrentes en el diseño de software. No son código copiable, sino plantillas de resolución que cada equipo adapta a su contexto. A continuación se investigan los patrones solicitados por la guía, organizados por categoría, y se determina cuáles son aplicables al stack Django + React.

Patrones creacionales

Factory (Fábrica). Proporciona una interfaz para crear objetos sin especificar sus clases concretas. En lugar de usar `new Clase()` directamente, se delega la creación a un método o clase fábrica que decide qué tipo de objeto instanciar según los parámetros recibidos.

Singleton (Instancia única). Garantiza que una clase tenga una sola instancia y proporciona un punto de acceso global a ella. Se usa cuando exactamente un objeto necesita coordinar acciones en todo el sistema, como una conexión a base de datos o una configuración global.

Builder (Constructor). Separa la construcción de un objeto complejo de su representación, de modo que el mismo proceso de construcción pueda crear diferentes representaciones. Es útil cuando un objeto tiene muchos parámetros opcionales o requiere varios pasos para configurarse.

Patrones estructurales

Adapter (Adaptador). Convierte la interfaz de una clase en otra interfaz que los clientes esperan. Permite que clases con interfaces incompatibles trabajen juntas envolviendo el objeto original en una clase adaptadora que traduce las llamadas.

Facade (Fachada). Proporciona una interfaz simplificada a un subsistema complejo. En lugar de que el cliente interactúe con múltiples clases del subsistema directamente, la fachada ofrece un único punto de entrada que delega las llamadas internamente.

Decorator (Decorador). Añade comportamiento dinámicamente a un objeto sin alterar su clase. Se envuelve el objeto original en una clase decoradora que implementa la misma interfaz pero añade funcionalidad antes o después de delegar al objeto envuelto.

Proxy (Apoderado). Proporciona un sustituto o marcador de posición para otro objeto para controlar el acceso a él. El proxy intercepta las llamadas al objeto real y puede añadir lógica adicional como validación, caché, control de acceso o retraso en la creación.

Patrones de comportamiento

Observer (Observador). Define una relación de uno a muchos: cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente. El objeto observado no necesita saber quién lo observa, lo que permite acoplar débilmente el emisor del receptor.

Strategy (Estrategia). Define una familia de algoritmos, encapsula cada uno y los hace intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo usan: en lugar de implementar la lógica con condicionales (if/elif/else), se crea una interfaz común y múltiples implementaciones que se pueden intercambiar en tiempo de ejecución.

Command (Comando). Encapsula una solicitud como un objeto, lo que permite parametrizar clientes con diferentes solicitudes, encolar solicitudes y registrar operaciones. Cada comando es un objeto con un método `execute()` que contiene la lógica de la acción.

Patrones arquitectónicos

Repository (Repositorio). Centraliza toda la lógica de acceso a datos de una entidad en una clase o conjunto de clases. El resto de la aplicación interactúa con el repositorio mediante una interfaz sin conocer detalles del ORM, el motor de base de datos o el lenguaje SQL subyacente.

Unit of Work (Unidad de Trabajo). Mantiene una lista de objetos afectados por una transacción de negocio y coordina la escritura de todos los cambios como una sola operación: si algo falla, todos los cambios se revierten. Garantiza que las operaciones de base de datos se ejecuten de forma atómica.

Dependency Injection (Inyección de dependencias). En lugar de que una clase cree sus dependencias internamente (usando `new` o llamadas directas), las dependencias se pasan desde fuera, generalmente a través del constructor o de métodos `setter`. Esto facilita las pruebas (se pueden inyectar objetos simulados) y reduce el acoplamiento entre clases.

Service Layer (Capa de servicios). Separa la lógica de negocio de las vistas y de los modelos. Las vistas reciben la petición, la delegan a un servicio que contiene la lógica, y el servicio coordina los repositorios y otros componentes necesarios para completar la operación.

CQRS (Command Query Responsibility Segregation). Separa las operaciones de escritura (comandos) de las operaciones de lectura (consultas) en modelos o interfaces diferentes. Esto permite optimizar cada lado de forma independiente: por ejemplo, las lecturas pueden ir a una base de datos replicada mientras las escrituras van a la principal.

Patrones aplicables al stack y su justificación

Al evaluar los 14 patrones frente a nuestra arquitectura con Django y React, la conclusión es muy clara: 13 de ellos encajan de forma natural en el proyecto (algunos provistos directamente por los frameworks y otros implementados por nosotros), mientras que solo uno, CQRS, se descarta con justa razón técnica. A continuación explicamos dónde, cómo y por qué aplica cada uno.

Factory. No hace falta reinventar la rueda en la aplicación principal, pero este patrón brilla en la capa de pruebas. Usamos la librería `factory_boy` para fabricar objetos de prueba al instante, evitando duplicar código de configuración en cada test unitario.

Singleton. Django ya lo aplica internamente sin que tengamos que programarlo: la configuración global (`settings`) y el pool de conexiones a la base de datos se mantienen como instancias únicas durante toda la ejecución. Entender que está ahí nos ayuda a evitar estados globales indeseados.

Builder. Es la esencia detrás de los `QuerySets` de Django. Cada vez que encadenamos `.filter()`, `.exclude()`, `.order_by()` o `.select_related()`, estamos construyendo paso a paso una consulta SQL que solo se ejecuta cuando realmente solicitamos los datos. Es imprescindible para la capa de persistencia.

Adapter. Es nuestro puente con servicios externos. Al comunicarnos con APIs de IA (como Gemini o OpenAI), sus respuestas tienen formatos ajenos a nuestro dominio. El adaptador traduce esa estructura al formato que esperan nuestros modelos, permitiendo cambiar de proveedor de IA sin romper el resto del sistema.

Facade. Los serializadores de Django REST Framework funcionan como la fachada ideal: simplifican y ocultan la complejidad de los modelos, sus relaciones y validaciones, exponiendo hacia el frontend únicamente un contrato JSON limpio y seguro.

Decorator. Lo aprovechamos en ambos extremos del stack. En el backend, usaremos decoradores como `@login_required`, `@permission_classes` o `@api_view` para añadir seguridad y contexto a las vistas sin ensuciar la lógica. En React, los custom Hooks y Higher-Order Components cumplen este mismo rol al empaquetar y reutilizar comportamiento.

Proxy. Trabaja a dos niveles clave. En infraestructura, Nginx actúa como proxy inverso protegiendo y redirigiendo peticiones hacia Django. En la lógica de negocio, nuestro backend actúa como proxy frente a las APIs de IA, evitando exponer claves secretas o tokens en el navegador del usuario.

Observer. Es el corazón de la reactividad. En Django, las signals nos permiten reaccionar a eventos (como enviar un correo de bienvenida justo al registrar un usuario) manteniendo el código totalmente desacoplado. En React, `useEffect` escucha los cambios de estado para actualizar la interfaz automáticamente.

Strategy. Nos da la libertad de alternar soluciones en tiempo de ejecución. En VocalCode, este patrón nos permite encapsular dos motores de interpretación de voz distintos (un diccionario técnico rígido y una IA generativa flexible) bajo una misma interfaz, de modo que el usuario cambie de motor sin alterar el flujo del editor.

Command. Ideal para convertir acciones en objetos manejables. En el ecosistema Django lo vemos en los comandos de gestión (`python manage.py...`). En la lógica del proyecto, cada instrucción dictada por voz se empaqueta como un Command con métodos `execute()` y `undo()`, abriendo la puerta a un historial de acciones y corrección por voz sin tocar el teclado.

Repository. Evita que las vistas o servicios dependan directamente de la sintaxis del ORM. Al centralizar las consultas de cada entidad en clases de repositorio (como `ProductoRepository`), logramos que la lógica de negocio no sufra si en el futuro decidimos cambiar el ORM o el motor de base de datos.

Unit of Work. Se manifiesta directamente a través de `transaction.atomic()` en Django. En lugar de crear una clase dedicada, agrupamos múltiples operaciones de base de datos dentro de este bloque: si algo falla en el camino, todo se revierte automáticamente, garantizando la integridad de los datos.

Dependency Injection. La clave para mantener un código fácil de probar. En lugar de instanciar servicios de IA o clientes HTTP directamente dentro de las vistas, los pasamos como parámetros o configuraciones. Esto nos permite inyectar "mocks" o servicios simulados durante las pruebas sin depender de APIs reales.

Service Layer. Es la cura para las "vistas gordas". En lugar de que una vista de DRF reciba la petición, valide datos, aplique reglas de negocio y consulte la base de datos en 100 líneas de código, la vista simplemente delega la tarea a una clase de servicio que coordina la lógica y devuelve el resultado.

CQRS (no aplicable). Separar las lecturas y las escrituras en modelos o bases de datos independientes añade una complejidad de sincronización enorme. Para un proyecto formativo desarrollado por un equipo pequeño, PostgreSQL y el ORM de Django gestionan ambas operaciones con total holgura. CQRS es excelente en sistemas gigantescos con millones de lecturas por segundo, pero aquí representaría sobreingeniería sin ningún beneficio real.

Fuentes consultadas en este bloque

- *Refactoring Guru — Design Patterns* — <https://refactoring.guru/design-patterns>
- *Django — QuerySets (patrón Builder)* — <https://docs.djangoproject.com/en/stable/topics/db/queries/>
- *Django — Decoradores HTTP (patrón Decorator)* — <https://docs.djangoproject.com/en/stable/topics/http/decorators/>
- *Django — Signals (patrón Observer)* — <https://docs.djangoproject.com/es/5.1/topics/signals/>
- *Django — Transacciones (Unit of Work)* — <https://docs.djangoproject.com/en/stable/topics/db/transactions/>
- *Django — Custom Management Commands (patrón Command)* — <https://docs.djangoproject.com/en/stable/howto/custom-management-commands/>
- *Django REST Framework — Serializers (patrón Facade)* — <https://www.django-rest-framework.org/api-guide/serializers/>
- *Martin Fowler — Repository Pattern* — <https://martinfowler.com/eaCatalog/repository.html>
- *Martin Fowler — CQRS* — <https://martinfowler.com/bliki/CQRS.html>

BLOQUE 6. PRINCIPIOS Y BUENAS PRÁCTICAS

Los principios de diseño de software son reglas generales que guían cómo escribir código que sea legible, mantenible y escalable. A diferencia de los patrones de diseño, que resuelven problemas estructurales específicos, los principios son lineamientos de alto nivel que aplican sin importar el lenguaje o el framework. En un stack que combina dos ecosistemas como Django y React, estos principios se materializan de forma distinta en cada capa, pero el objetivo es el mismo: que cualquier desarrollador pueda leer el código, entenderlo y modificarlo sin temor a romper algo.

Principios

SOLID. Es un acrónimo de cinco principios de diseño orientado a objetos formulados por Robert C. Martin. Single Responsibility establece que una clase debe tener una sola razón para cambiar. Open/Closed indica que el código debe estar abierto a la extensión pero cerrado a la modificación. Liskov Substitution exige que las clases hijas puedan sustituir a sus padres sin alterar el programa. Interface Segregation recomienda interfaces pequeñas y específicas en lugar de una general. Dependency Inversion indica que los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones.

DRY (Don't Repeat Yourself). Establece que cada pieza de conocimiento debe tener una representación única y no autoritaria. Cuando el mismo código aparece en dos o más lugares, cualquier cambio futuro requiere modificar todos esos lugares, y si se olvida uno, el sistema queda inconsistente. Es uno de los principios fundacionales de Django.

KISS (Keep It Simple, Stupid). Establece que la solución más simple que funciona es casi siempre mejor que la solución ingeniosa. No se trata de escribir código malo, sino de no sobre-diseñar: la complejidad se añade solo cuando el problema lo requiere, no de forma preventiva.

YAGNI (You Aren't Gonna Need It). Complementa a KISS y establece que no se deben construir funcionalidades que no se necesitan en el momento presente, por mucho que parezcan que podrían ser útiles en el futuro. El código que no se usa es código que hay que mantener, probar y entender, y que añade complejidad sin aportar valor.

Separation of Concerns (Separación de intereses). Establece que cada componente del sistema debe tener una responsabilidad bien definida y no mezclar responsabilidades que pertenecen a distintas capas. Es la base misma de tener un backend y un frontend separados: cada uno se ocupa de lo suyo sin invadir el territorio del otro.

Clean Code. No es un principio único sino una colección de prácticas que hacen que el código sea legible para humanos: nombres descriptivos, funciones cortas que hacen una sola cosa, comentarios que explican el "por qué" no el "qué", y un formato consistente en todo el proyecto. La idea es que el código se lea como documentación, no como un acertijo que hay que descifrar.

Aplicación en el desarrollo del proyecto

A continuación se explica cómo cada principio se concreta en el desarrollo de VocalCode, el editor de código controlado por voz.

SOLID. En el backend, Single Responsibility se aplica al separar la lógica de interpretación de voz (servicio InterpretacionService) de la persistencia de proyectos (repositorio ProyectoRepository) y de la exposición vía API (vista DictadoView). Open/Closed permite agregar un nuevo proveedor de IA (por ejemplo, cambiar de Gemini a un modelo local) creando un nuevo adaptador sin modificar el servicio de interpretación. Liskov Substitution garantiza que cualquier estrategia de interpretación (diccionario técnico o IA) pueda usarse de forma intercambiable. Interface Segregation se refleja en que el serializador de dictado solo incluye los campos necesarios para esa operación, no todos los campos del modelo. Dependency Inversion se aplica al inyectar el proveedor de IA en el servicio de interpretación en lugar de que el servicio lo cree directamente, lo que permite usar un mock en los tests.

DRY. En el backend, el vocabulario técnico de programación (mapeo de "abre paréntesis" al carácter "(") se define una sola vez en un diccionario centralizado y todas las instrucciones de voz lo consultan, en lugar de tener la traducción dispersa en múltiples archivos. En el frontend, el componente del editor de código se construye una vez y se reutiliza en todas las pantallas donde el usuario dicta. El hook que maneja el reconocimiento de voz se extrae como un custom Hook en lugar de repetir la lógica de Web Speech API en cada componente que la necesite.

KISS. El flujo de dictado no requiere un motor de procesamiento de lenguaje natural complejo: se usa un diccionario directo que mapea frases de voz a acciones concretas ("abre paréntesis" genera "("), y solo cuando el usuario lo necesite se activa la IA generativa como modo alternativo. La autenticación se maneja con SimpleJWT de DRF, que ya resuelve el problema sin necesidad de construir un sistema propio. El frontend usa React con Context API para el estado del usuario en lugar de agregar Redux, que sería sobre-diseño para la cantidad de estado que maneja la aplicación.

YAGNI. No se construirá un sistema de roles complejo con múltiples niveles jerárquicos (superadmin, admin, editor, viewer, etc.): el MVP necesita dos roles (usuario que dicta código y administrador que gestiona el sistema). No se implementará un sistema de plugins o extensiones para el editor: las funcionalidades que se necesitan están definidas y no hay margen para que el usuario agregue las suyas en esta fase. No se configurará un pipeline de CI/CD con múltiples etapas (lint, test, build, deploy automático) porque el equipo despliega manualmente desde GitHub según lo necesite.

Separation of Concerns. React se ocupa exclusivamente de capturar la voz del usuario mediante Web Speech API, mostrar el editor con resaltado de sintaxis (CodeMirror) y reflejar las inserciones que llegan desde el backend. Django se ocupa de recibir la transcripción de voz, interpretarla usando el diccionario técnico o la IA, y persistir el código generado en PostgreSQL. Ninguno de los dos accede directamente al micrófono ni a la base de datos del otro: la comunicación es siempre a través de la API REST.

Clean Code. Las clases del backend llevan nombres descriptivos: VoiceInterpreter, DictationLog, CommandInterpreter en lugar de Procesador1, Log1, Comando. Las funciones se mantienen cortas: la función que traduce una instrucción de voz a código no debería superar las 20 líneas; si lo hace, se extrae parte de la lógica a funciones auxiliares. Los comentarios se usan para explicar decisiones de diseño (por qué se eligió PostgreSQL sobre MySQL, por qué se usa un diccionario y no solo IA), no para describir lo que el código ya dice. Black formatea el código Python y Prettier + ESLint el código JavaScript/JSX, ambos integrados como hooks de pre-commit en Git para que el código que llega al reposio ya cumpla el estándar sin depender de que cada desarrollador lo aplique manualmente.

Fuentes consultadas en este bloque

- Documentación oficial de Django — Filosofías de diseño (DRY, Acoplamiento débil) — <https://docs.djangoproject.com/es/5.1/misc/design-philosophies/>

- Refactoring Guru — Guía de Principios SOLID en español — <https://refactoring.guru/es/solid>
- Martin Fowler — Bliki: YAGNI (You Aren't Gonna Need It) — <https://martinfowler.com/bliki/Yagni.html>
- Ruff Docs — Linter y Formateador oficial para Python (Clean Code) — <https://docs.astral.sh/ruff/>
- Prettier — Formateador de código para React y JS — <https://prettier.io/>
- ESLint — Linter y análisis estático para JavaScript y React — <https://eslint.org/>

BLOQUE 7. SEGURIDAD

La seguridad en una aplicación web abarca desde la autenticación de usuarios hasta la protección del código contra ataques conocidos. En un stack desacoplado como Django + React, la superficie de ataque es mayor que en una aplicación monolítica porque el frontend y el backend se comunican a través de HTTP, lo que expone endpoints que pueden ser consumidos por clientes no autorizados si no se configuran correctamente las políticas de acceso entre dominios.

Autenticación, autorización y roles

Autenticación. Es el proceso de verificar la identidad de un usuario: confirmar quién es. En una API REST como la que expone Django, esto implica recibir credenciales (usuario y contraseña), validarlas contra la base de datos y devolver un token o sesión que el cliente usará en las peticiones siguientes.

Autorización. Es el proceso de determinar qué puede hacer un usuario una vez identificado. No se trata de saber quién es, sino de saber si tiene permiso para acceder a un recurso o ejecutar una acción. En Django, el sistema de permisos (permissions) y la propiedad de los objetos (object-level permissions) controlan qué puede hacer cada usuario con cada dato.

Roles. Es una forma de agrupar permisos bajo un nombre que representa una función dentro del sistema (por ejemplo, "administrador", "editor", "usuario estándar"). Django no incluye un sistema de roles nativo, pero se puede implementar mediante grupos de permisos: cada rol es un grupo con un conjunto predefinido de permisos, y los usuarios se asignan a uno o más grupos.

JWT y OAuth

JWT (JSON Web Token). Es un estándar abierto (RFC 7519) para representar de forma compacta y verificable la identidad de un usuario y sus permisos. Un JWT consta de tres partes codificadas en base64: un header (algoritmo de firma), un payload (datos del usuario como ID, roles, fecha de expiración) y una signature (firma digital generada con la clave secreta del servidor). El token viaja en cada petición del frontend al backend sin que el servidor necesite consultar la base de datos para verificar quién es el usuario.

OAuth 2.0. Es un protocolo de autorización que permite a un usuario conceder acceso a su cuenta en un servicio a un tercero sin compartir su contraseña. Es el mecanismo que usan los botones "Iniciar sesión con Google" o "Iniciar sesión con GitHub": la aplicación redirige al usuario al proveedor, el

proveedor devuelve un código de autorización, y el backend lo intercambia por un token de acceso. No sustituye a JWT sino que lo complementa como forma de obtener el primer token sin manejar contraseñas propias.

Gestión de sesiones y hashing

Gestión de sesiones. En una aplicación web tradicional, el servidor crea una sesión en memoria o en base de datos tras el login y envía una cookie con el ID de sesión al navegador. En cada petición posterior, el navegador envía la cookie y el servidor la usa para identificar al usuario. En una API con JWT, este mecanismo no se necesita porque el propio token contiene la información del usuario, lo que facilita escalar el backend horizontalmente porque cualquier servidor puede verificar el token sin consultar una sesión compartida.

Hashing. Es el proceso de transformar una contraseña en una cadena fija de longitud fija mediante una función matemática de un solo sentido: se puede verificar que una contraseña coincida con el hash almacenado, pero no se puede recuperar la contraseña original a partir del hash. Django usa PBKDF2 por defecto con 390,000 iteraciones de HMAC-SHA256, y permite cambiar a Argon2 o bcrypt, que son más resistentes a ataques de fuerza bruta con hardware moderno. Django nunca almacena contraseñas en texto plano.

CORS y CSRF

CORS (Cross-Origin Resource Sharing). Es un mecanismo de seguridad del navegador que restringe qué dominios pueden hacer peticiones a un servidor. Cuando React corre en localhost:3000 y Django en localhost:8000, el navegador bloquea las peticiones de React a Django porque son dominios distintos (orígenes cruzados). La librería django-cors-headers permite configurar en Django qué orígenes están permitidos, qué métodos HTTP acepta y qué cabeceras puede enviar. Sin esta configuración, la API simplemente no funciona desde el frontend.

CSRF (Cross-Site Request Forgery). Es un ataque en el que un sitio malicioso engaña al navegador de un usuario autenticado para que envíe una petición a un servidor en el que el usuario ya tiene sesión, aprovechando que el navegador adjunta automáticamente las cookies de sesión. Django incluye protección CSRF activada por defecto para formularios HTML: cada formulario lleva un token oculto que el servidor verifica al recibir el envío. En una API con JWT, el riesgo de CSRF se reduce porque el token no se envía automáticamente como cookie sino que el frontend lo adjunta manualmente en el header Authorization, pero la protección sigue activa si se usan cookies para almacenar el token.

XSS, inyección SQL e inyección NoSQL

XSS (Cross-Site Scripting). Es un ataque que inyecta código JavaScript malicioso en una página web para que se ejecute en el navegador de otros usuarios. React reduce este riesgo porque escapa automáticamente el contenido que se renderiza en el DOM: si un texto contiene la etiqueta script de

HTML, React lo muestra como texto literal, no como código ejecutable. Sin embargo, la protección se anula si se usa `dangerouslySetInnerHTML` en React, que permite insertar HTML raw sin sanitización, o si el backend devuelve contenido usuario sin escapar y el frontend lo renderiza con `dangerouslySetInnerHTML`.

Inyección SQL. Es un ataque que inserta comandos SQL maliciosos en campos de entrada para manipular la base de datos. El ORM de Django protege contra esto porque parametriza automáticamente todas las consultas: en lugar de concatenar strings para construir SQL, envía los parámetros por separado y el driver de base de datos los escapa. La protección se pierde si se usa `.raw()` o `.extra()` con cadenas formateadas por el usuario sin parametrizar.

Inyección NoSQL. Es el equivalente para bases de datos NoSQL: inyectar operadores de consulta maliciosos en campos de entrada. Aunque este stack usa PostgreSQL (relacional), si se usan campos JSONB del ORM, es importante no interpolar datos de usuario directamente en consultas raw y siempre usar los lookups que el ORM proporciona, que escapan los valores automáticamente.

HTTPS, variables de entorno y gestión de secretos

HTTPS. Es el protocolo HTTP sobre una capa de encriptación TLS/SSL. Sin HTTPS, toda la comunicación entre el navegador y el servidor viaja en texto plano, lo que permite a cualquier intermediario en la red leer el contenido de las peticiones (incluyendo tokens JWT, contraseñas y datos personales). En producción, HTTPS es obligatorio, no opcional. Los proveedores como Render, Vercel y Netlify lo incluyen de forma gratuita mediante certificados de Let's Encrypt.

Variables de entorno y gestión de secretos. Las variables de entorno son valores de configuración que se cargan desde el sistema operativo en lugar de escribirse directamente en el código. En este stack, la clave secreta de Django (`SECRET_KEY`), las credenciales de base de datos y las API keys de los proveedores de IA se definen como variables de entorno y se cargan con `python-dotenv` en el backend y con archivos `.env` en el frontend. Estas variables nunca se suben al repositorio de código: se incluye un archivo `.env.example` con valores placeholder y se agrega `.env` a `.gitignore`, de modo que cada desarrollador configure sus propias credenciales localmente sin exponer las del equipo.

Vulnerabilidades más comunes del stack

Basado en las características propias de Django + React, estas son las vulnerabilidades más frecuentes en este stack:

- Exposición de la clave secreta (`SECRET_KEY`) o de credenciales de base de datos y APIs en el repositorio de código, ya sea por cometer un `.env` real o por incluirlo accidentalmente en un commit.

- Configuración de CORS demasiado permisiva (`ALLOWED_ORIGINS = ["*"]`) que permite que cualquier dominio consuma la API, no solo el frontend autorizado.
- Tokens JWT sin expiración adecuada o almacenados en `localStorage` sin protecciones adicionales, lo que los hace vulnerables a robo si hay XSS en el frontend.
- Falta de validación de permisos a nivel de objeto en DRF: un usuario autenticado podría acceder a recursos de otro usuario simplemente cambiando el ID en la URL si no se implementa la validación en el queryset de la vista (por ejemplo, usando `get_queryset().filter(user=request.user)`).
- Dependencias desactualizadas en `requirements.txt` o `package.json` que contienen vulnerabilidades conocidas, detectables con herramientas como `pip-audit` o `npm audit`.
- Uso de `dangerouslySetInnerHTML` en React con contenido proveniente del usuario sin sanitizar, lo que abre la puerta a ataques XSS.
- Consultas al ORM con `.raw()` o `.extra()` que concatenan datos de usuario sin usar parámetros, abriendo la puerta a inyección SQL.

Fuentes consultadas en este bloque

- OWASP Top 10 — Proyecto Oficial de Riesgos de Seguridad en Aplicaciones Web — <https://owasp.org/www-project-top-ten/>
- Documentación Oficial de Django 6.0 — Resumen de Seguridad (XSS, SQLi, CSRF) — <https://docs.djangoproject.com/es/6.0/topics/security/>
- Documentación Oficial de Django 6.0 — Gestión de Contraseñas y Hashing — <https://docs.djangoproject.com/es/6.0/topics/auth/passwords/>
- Documentación Oficial de Django 6.0 — Protección CSRF — <https://docs.djangoproject.com/es/6.0/ref/csrf/>
- Django REST Framework — Guía de Autenticación y Permisos — <https://www.django-rest-framework.org/api-guide/authentication/>
- `django-cors-headers` — Repositorio Oficial y Configuración CORS — <https://github.com/adamchainz/django-cors-headers>
- JWT.io — Introducción Estándar a JSON Web Tokens (RFC 7519) — <https://jwt.io/introduction>
- Documentación Oficial de React — Prevención de XSS y `dangerouslySetInnerHTML` — <https://react.dev/reference/react-dom/components/common#dangerouslysetinnerhtml>

BLOQUE 8. DATOS Y PERSISTENCIA

La persistencia de datos es la capacidad de un sistema para almacenar información de forma permanente, es decir, que los datos sobrevivan a la ejecución del programa. En un stack como Django + React, la persistencia recae enteramente en el backend: Django define los modelos de datos, interactúa con el motor de base de datos a través de su ORM, y React no toca la base de datos directamente sino que consume los datos a través de la API REST. El frontend puede cachear datos en memoria para mejorar la experiencia de usuario, pero la fuente oficial de verdad es siempre el backend.

Modelo de datos

Qué es. Un modelo de datos es la representación abstracta de las entidades del dominio del problema y las relaciones entre ellas. No es la base de datos física, sino el diseño conceptual: qué entidades existen (usuarios, proyectos, instrucciones de voz), qué atributos tiene cada una y cómo se relacionan entre sí. Este diseño se materializa en Django como clases Python que heredan de `models.Model`, y el ORM se encarga de traducir esas clases a tablas concretas en la base de datos.

SQL y NoSQL

SQL (Structured Query Language). Es el lenguaje estándar para definir, manipular y consultar datos en bases de datos relacionales. Los datos se organizan en tablas con filas y columnas de tipos fijos, y las relaciones entre tablas se definen mediante llaves foráneas y restricciones de integridad. El modelo relacional prioriza la consistencia y la estructura por sobre la flexibilidad.

NoSQL. Es un término amplio que agrupa bases de datos no relacionales: documentales (MongoDB), clave-valor (Redis), de grafos (Neo4j), entre otras. Estas bases de datos sacrifican algunas garantías de consistencia relacional a cambio de flexibilidad en el modelo de datos, escalabilidad horizontal más sencilla y la capacidad de manejar datos no estructurados. Este stack utiliza un modelo relacional porque la naturaleza del proyecto (editor de código con usuarios, proyectos, historial de acciones) se adapta mejor a una estructura tabular con relaciones bien definidas que a documentos flexibles sin estructura fija.

ORM y ODM

ORM (Object-Relational Mapping). Es una técnica que traduce objetos de un lenguaje de programación a filas de una base de datos relacional, y viceversa. El ORM de Django permite definir modelos como clases de Python y operar sobre ellos usando sintaxis Python en lugar de escribir SQL manual. Por ejemplo, `Producto.objects.filter(precio__gt=100)` genera automáticamente la consulta SQL equivalente sin que el desarrollador escriba una sola línea de SQL.

ODM (Object-Document Mapping). Es el equivalente del ORM pero para bases de datos documentales como MongoDB: traduce objetos a documentos JSON. Este stack no usa ODM porque la base de datos es relacional (PostgreSQL), pero si se usara MongoDB con Django, la librería `MongoEngine` cumpliría esta función. La distinción es importante porque cada enfoque tiene ventajas y limitaciones distintas que afectan la elección del stack.

Migraciones

Las migraciones son el mecanismo que permite versionar los cambios en el modelo de datos a lo largo del tiempo. Cada vez que se agrega un campo a un modelo, se modifica una relación o se cambia un tipo de dato, Django genera automáticamente un archivo de migración que describe la

operación necesaria para actualizar el esquema de la base de datos. Estos archivos se ejecutan secuencialmente: cada uno tiene un número de orden y Django registra cuáles ya se aplicaron, de modo que puede avanzar o revertir a cualquier punto del historial del esquema sin perder datos. Sin migraciones, los cambios en los modelos tendrían que aplicarse manualmente con SQL, lo que sería propenso a errores y imposible de rastrear en un equipo.

Relaciones

Las relaciones definen cómo se conectan las entidades entre sí. Django soporta tres tipos mediante campos especializados del ORM: uno a uno (`OneToOneField`, por ejemplo, extender el modelo de usuario por defecto con datos adicionales), uno a muchos (`ForeignKey`, por ejemplo, un proyecto pertenece a un usuario) y muchos a muchos (`ManyToManyField`, por ejemplo, un proyecto puede tener múltiples etiquetas y una etiqueta puede pertenecer a múltiples proyectos). Estas relaciones se reflejan en la base de datos como llaves foráneas y tablas intermedias, y el ORM permite navegarlas usando notación de puntos: `proyecto.usuario.nombre` accede al usuario propietario del proyecto sin escribir joins SQL explícitos.

Índices

Un índice es una estructura de datos auxiliar que la base de datos crea sobre una o varias columnas para acelerar las consultas. Sin índices, una consulta con `WHERE` sobre una columna sin indexar requiere recorrer toda la tabla fila por fila (full table scan). Con un índice, la base de datos puede ir directamente a las filas que cumplen la condición, reduciendo el tiempo de consulta drásticamente. Django permite definir índices mediante `class Meta: indexes = [...]` en cualquier modelo, y la base de datos los crea automáticamente al ejecutar las migraciones. El costo de los índices es que cada insert, actualización y eliminación es ligeramente más lento, porque la base de datos debe actualizar también el índice correspondiente.

Transacciones e integridad

Una transacción agrupa varias operaciones de base de datos en una unidad atómica: o todas se ejecutan correctamente, o ninguna se ejecuta. Si la tercera operación de un grupo falla, las dos anteriores se revierten automáticamente y la base de datos queda en un estado consistente, como si nada hubiera pasado. Esto evita datos corruptos como un registro de venta sin su correspondiente pedido, o un descuento aplicado a un producto sin registro del descuento. En Django, `transaction.atomic()` envuelve un bloque de código en una transacción: si se lanza una excepción dentro del bloque, todas las operaciones de base de datos se revierten. La integridad referencial, por su parte, es la garantía de que las relaciones entre tablas siempre sean válidas: no puede existir un proyecto que referencie a un usuario que no existe, gracias a las restricciones de llave foránea que la base de datos impone.

Backup y escalabilidad

El backup consiste en crear copias de seguridad de la base de datos que permitan recuperar el sistema en caso de fallo del hardware, corrupción de datos o error humano como eliminar registros por accidente. PostgreSQL ofrece `pg_dump` para backups lógicos y `point-in-time recovery` para recuperar el estado de la base de datos en un momento específico del pasado. MySQL ofrece `mysqldump` con una función similar. La escalabilidad se refiere a la capacidad del sistema para manejar más carga de trabajo a medida que crece la demanda. En una base de datos relacional, la escalabilidad vertical (añadir más recursos al mismo servidor: más CPU, más RAM, mejor disco) es directa. La escalabilidad horizontal (añadir más servidores de base de datos y distribuir la carga entre ellos) es más compleja porque requiere replicación de datos, coordinación de escrituras y manejo de conflictos. Para la mayoría de aplicaciones, incluido este proyecto, la escalabilidad vertical con PostgreSQL es suficiente.

Justificación de la base de datos elegida

De las dos opciones que recomienda el stack (PostgreSQL y MySQL), se selecciona PostgreSQL porque resuelve mejor los requerimientos específicos del proyecto. El sistema necesita almacenar vocabularios personalizados por usuario: cada usuario añade sus propias frases y mapeos de código. PostgreSQL tiene JSONB nativo con índices GIN que permiten buscar dentro de esos vocabularios de forma rápida, mientras que MySQL guarda JSON como texto plano sin índices nativos para búsquedas dentro del JSON, lo que se vuelve lento a medida que el diccionario crece.

El motor de dictado también requiere búsquedas instantáneas de frases habladas en el diccionario técnico. PostgreSQL incluye full-text search nativo con soporte para búsquedas por similitud, prefijos y frases, mientras que en MySQL esa funcionalidad es más limitada y requiere configuraciones adicionales con plugins como MySQL Full-Text Search. Además, el ORM de Django está optimizado para PostgreSQL: campos como `ArrayField` (listas), `RangeField` (rangos de fechas o números), `JSONField` (datos semiestructurados) y lookups avanzados como `__icontains`, `__regex`, `__search` funcionan de inmediato, mientras que en MySQL no están disponibles o requieren reimplementarse en código Python.

Finalmente, PostgreSQL es el motor que la documentación oficial de Django recomienda para producción y el que la comunidad Django considera el estándar de facto, lo que garantiza mejor soporte, más ejemplos en la documentación y mayor estabilidad a largo plazo. Todos estos factores hacen que PostgreSQL sea la elección con mejor relación costo-beneficio para un stack Django + React que necesita manejar datos semiestructurados, búsquedas en vocabularios y potencial integración con capacidades de inteligencia artificial.

Fuentes consultadas en este bloque

- Documentación Oficial de Django 6.0 — Modelos, Relaciones e Índices — <https://docs.djangoproject.com/es/6.0/topics/db/models/>
- Documentación Oficial de Django 6.0 — Control de Versiones de Esquema y Migraciones — <https://docs.djangoproject.com/es/6.0/topics/migrations/>
- Documentación Oficial de Django 6.0 — Manejo de Transacciones y Atomicidad (transaction.atomic) — <https://docs.djangoproject.com/es/6.0/topics/db/transactions/>
- Documentación Oficial de Django 6.0 — Módulo Específico para PostgreSQL (JSONField, ArrayField, Full-Text Search) — <https://docs.djangoproject.com/es/6.0/ref/contrib/postgres/>
- PostgreSQL Global Development Group — Características Generales de PostgreSQL — <https://www.postgresql.org/about/>
- PostgreSQL Documentation — Herramienta de RespalDOS y Backups Lógicos (pg_dump) — <https://www.postgresql.org/docs/current/app-pgdump.html>

BLOQUE 9. DESARROLLO PROFESIONAL

Un stack tecnológico no se limita al código: también implica un flujo de trabajo profesional que permite a un equipo colaborar sin pisarse el trabajo entre sí y sin romper lo que ya funciona. Esto incluye controlar el historial de cambios, coordinar el trabajo en paralelo mediante ramas, revisar el código antes de integrarlo, gestionar las dependencias del proyecto de forma reproducible, y escribir pruebas que den confianza al equipo de que el sistema funciona correctamente incluso cuando se hacen cambios.

Git y GitHub/GitLab

Git. Es un sistema de control de versiones distribuido que registra instantáneas (commits) del estado de un proyecto en cada punto de su evolución. Cada commit guarda qué cambió, quién lo cambió, cuándo y por qué. A diferencia de herramientas centralizadas como SVN, Git guarda una copia completa del repositorio con todo su historial en la máquina de cada desarrollador, lo que permite trabajar sin conexión a un servidor central y resolver conflictos de forma local.

GitHub y GitLab. Son plataformas en la nube que alojan repositorios Git y agregan funcionalidades colaborativas sobre ese base: interfaz web para visualizar el código y su historial, gestión de ramas con pull/merge requests, herramientas de revisión de código, seguimiento de issues, wikis de documentación del proyecto e integración continua y despliegue automatizado. GitHub es la plataforma más usada a nivel global; GitLab puede instalarse en servidores propios, lo que lo hace común en empresas que necesitan mantener el código de forma privada.

Branching (ramas)

Las ramas permiten crear líneas paralelas de desarrollo aisladas que no afectan la rama principal. La rama principal (main o master) contiene el código estable que está en producción. Cada funcionalidad nueva se desarrolla en una rama separada (por ejemplo, feature/dictado-por-voz), y

cuando está lista y probada, se integra a la rama principal mediante un pull request. Esto permite que varios desarrolladores trabajen en distintas funcionalidades al mismo tiempo sin generar conflictos, y que la rama principal siempre contenga código que se sabe que funciona.

Code Review

Es el proceso de leer y analizar el código escrito por otro miembro del equipo antes de integrarlo al proyecto. Se realiza a través de pull requests: el autor de la rama crea la solicitud, un revisor lee los cambios, deja comentarios específicos sobre errores, estilo o mejoras posibles, y aprueba o solicita modificaciones. El objetivo es detectar problemas antes de que lleguen a producción, compartir conocimiento entre los miembros del equipo y mantener un estándar de calidad en todo el código del proyecto. No se trata de buscar la perfección, sino de asegurar que el código sea comprensible, que siga las convenciones del equipo y que no introduzca regresiones.

Gestión de dependencias

La gestión de dependencias consiste en listar, versionar y actualizar las librerías que el proyecto necesita para funcionar. En el backend de Django, pip lee el archivo requirements.txt (o Poetry lee pyproject.toml) e instala las librerías desde PyPI junto con sus versiones exactas, lo que garantiza que cualquier desarrollador pueda recrear el mismo entorno de desarrollo ejecutando un solo comando. En el frontend de React, npm o yarn leen package.json e instalan los paquetes desde el registro npm con sus versiones fijas en package-lock.json o yarn.lock. Fijar las versiones es fundamental: si dos desarrolladores instalan versiones distintas de una misma librería, el proyecto puede comportarse de forma distinta en cada máquina, lo que genera el clásico "en mi máquina sí funciona".

Tipos de pruebas

Pruebas unitarias. Verifican una unidad aislada de código en particular: una función, un método, un componente de React o un serializador de Django. Se ejecutan de forma aislada, sin conectar base de datos ni otros sistemas. En Django se escriben con el módulo de testing o con pytest-django, y en React con Jest. Ejemplo: verificar que un serializador rechace un dato inválido, o que un componente de React renderiza correctamente cierta información cuando recibe ciertas props.

Pruebas de integración. Verifican que varios componentes trabajan correctamente cuando se conectan entre sí. Ya no están aislados: pueden involucrar la base de datos, el sistema de archivos, o la comunicación entre módulos. Ejemplo: verificar que una vista de Django efectivamente guarda un registro en PostgreSQL y que los datos se pueden recuperar después, o que un componente de React consume correctamente la API del backend sin errores.

Pruebas funcionales. Verifican que una funcionalidad completa cumple con lo esperado desde la perspectiva del usuario, sin entrar en detalles de implementación interna. No prueban cómo funciona el sistema internamente, sino que el resultado final es el correcto. Ejemplo: verificar que el flujo completo de login (ingresar usuario y contraseña, recibir un token, usarlo para acceder a una ruta protegida, y cerrar sesión) funciona de extremo a extremo.

Pruebas E2E (End to End). Simulan el flujo completo de un usuario real navegando por la aplicación React y consumiendo la API real del backend. A diferencia de las pruebas funcionales, las E2E operan sobre la aplicación desplegada (o en un entorno de pruebas que la replica lo más fielmente posible), incluyendo el navegador real. Se usan herramientas como Cypress o Playwright para automatizar las interacciones (hacer clic en un botón, llenar un formulario, verificar que aparezca un resultado esperado en pantalla). Son las pruebas más lentas y frágiles (un cambio en la interfaz puede romper una prueba E2E aunque la lógica del backend siga siendo correcta), pero son las que más se acercan a lo que el usuario final experimenta.

Fuentes consultadas en este bloque

- Documentación Oficial de Django 6.0 — Automatización y Ejecución de Pruebas (Testing) — <https://docs.djangoproject.com/es/6.0/topics/testing/>
- React Testing Library — Guía Oficial para Pruebas de Componentes en React — <https://testing-library.com/docs/react-testing-library/intro/>
- Git SCM — Documentación Oficial del Sistema de Control de Versiones — <https://git-scm.com/doc>
- GitHub Docs — Flujo de Trabajo con Ramas, Pull Requests y Code Review — <https://docs.github.com/es/pull-requests>
- Python Packaging Authority — Guía de Gestión de Dependencias (pip y PyPI) — <https://packaging.python.org/>
- Playwright Docs — Automatización de Pruebas Extremo a Extremo (E2E) — <https://playwright.dev/>

BLOQUE 10. DESPLIEGUE Y CLOUD

Una vez construida la aplicación, necesita ponerse a disposición de los usuarios. La decisión de dónde y cómo desplegar afecta la disponibilidad, el rendimiento, los costos y la complejidad operativa del proyecto. En un stack desacoplado como Django + React, el frontend y el backend se pueden desplegar de forma independiente en plataformas distintas, lo que ofrece flexibilidad pero requiere configurar la comunicación entre ambos para que la API funcione en producción.

Desarrollo local

Es la fase donde ambos proyectos corren en la máquina del desarrollador para escribir y probar código. Django se ejecuta con su servidor de desarrollo (`python manage.py runserver`), React con el suyo (`npm run dev`), y se comunican entre sí en distintos puertos del localhost. En esta fase no importa la velocidad ni la disponibilidad: lo importante es que sea rápido de configurar, que permita depurar, y que refleje fielmente el entorno de producción. Django y React ofrecen servidores de desarrollo que recargan automáticamente ante cada cambio, lo que acelera el ciclo de desarrollo.

Hosting, VPS y Cloud

Hosting. Es un servicio que proporciona un servidor listo para publicar un sitio web. El proveedor gestiona el servidor operativo, las actualizaciones de seguridad y la disponibilidad, pero el desarrollador tiene control limitado sobre la configuración del entorno. Es la opción más sencilla pero la menos flexible: típico de alojamientos compartidos donde varios proyectos comparten un mismo servidor con recursos limitados.

VPS (Servidor Privado Virtual). Es un servidor virtual dedicado que el desarrollador administra con acceso completo al sistema operativo (generalmente Linux). Ofrece control total sobre el entorno: se puede instalar cualquier versión de Python, PostgreSQL, Redis, Nginx y configurar todo a medida. Es la opción más flexible pero la que más tiempo de mantenimiento exige: el equipo es responsable de actualizar el sistema, aplicar parches de seguridad, configurar firewalls y resolver cualquier problema de infraestructura.

Cloud (Nube). Es la entrega de servicios de cómputo a través de internet bajo demanda. Los proveedores como AWS, Azure y Google Cloud ofrecen servicios administrados en distintas capas: infraestructura como servicio (IaaS, se alquila un servidor virtual y se gestiona todo lo demás), plataforma como servicio (PaaS, se despliega el código y el proveedor gestiona el sistema operativo y el runtime) y software como servicio (SaaS, se consume software ya terminado como GitHub o Gmail sin gestionar infraestructura). La ventaja principal es la escalabilidad bajo demanda: se pagan solo los recursos que se usen, no un servidor fijo que queda subutilizado cuando hay poco tráfico.

IaaS, PaaS y SaaS

Estos tres modelos representan distintos niveles de abstracción en la nube. IaaS ofrece infraestructura bruta (servidores virtuales, redes, almacenamiento) y el equipo administra todo lo demás: es lo más flexible pero el que más requiere conocimiento operativo. PaaS administra el sistema operativo y el entorno de ejecución y el equipo solo despliega su código: es el punto intermedio entre flexibilidad y simplicidad, ideal para proyectos como este, donde Render o Railway permiten desplegar Django + PostgreSQL con pocos clics sin configurar servidores. SaaS es software ya terminado que se consume directamente: el equipo no gestiona infraestructura ni despliegues, solo usa el servicio tal como GitHub para control de versiones o SendGrid para correos transaccionales.

Docker

Docker es una herramienta que empaqueta una aplicación junto con todo lo que necesita para funcionar (dependencias, configuración, sistema operativo base) en un contenedor estándar que se ejecuta igual en cualquier entorno. En este stack, se crea un contenedor para el backend Django con su entorno Python, otro para PostgreSQL, y un tercero para Redis si se usa como caché y broker de

Celery. Docker Compose permite definir todos estos contenedores en un archivo YAML y levantarlos todos con un solo comando (`docker-compose up`), eliminando el problema de "en mi máquina sí funciona". En producción, los mismos contenedores que se usan en desarrollo se despliegan en la nube, garantizando que el comportamiento sea idéntico en ambos entornos.

Serverless

Es un modelo donde el proveedor de nube ejecuta el código solo cuando se necesita, sin que el desarrollador administre servidores. Se cobra por cada ejecución, no por tiempo alquilado. En este stack, serverless se adapta bien al frontend estático de React: plataformas como Vercel y Netlify construyen y distribuyen la aplicación React automáticamente desde GitHub, con edge caching (copias de la aplicación en servidores distribuidos globalmente para reducir la latencia de carga). Sin embargo, Django no es nativo de serverless: mantiene conexiones persistentes a la base de datos, requiere un proceso corriendo continuamente, y forzarlo con herramientas como Zappa introduce cold starts (tiempo de arranque en frío la primera petición) y limitaciones de tamaño que degradan la experiencia del usuario, especialmente en una aplicación de tiempo real como un editor de código controlado por voz.

Plataformas evaluadas

Para el backend de Django se evaluaron plataformas que soporten Python nativamente y ofrezcan PostgreSQL administrado. Para el frontend de React, plataformas especializadas en despliegue de aplicaciones React estáticas. A continuación se presenta la evaluación de cada una.

Backend: Render. Ofrece free tier con despliegue automático desde GitHub, PostgreSQL administrado con respaldo automático, SSL gratuito, soporte nativo de WebSockets (necesario para tiempo real) y configuración mínima: se conecta el repositorio, se selecciona la rama principal y Render despliega automáticamente. Es la opción más recomendada para un MVP académico porque no requiere configuración de servidores ni conocimiento de infraestructura.

Backend: Railway. Alternativa similar a Render con free tier, PostgreSQL administrado, despliegue automático desde GitHub y configuración sencilla. Sin embargo, su plan gratuito tiene límites más restrictivos para conexiones activas de WebSockets, lo que puede ser un problema en una aplicación de tiempo real que mantiene conexiones persistentes. Adecuada como segunda opción o para prototipos que no requieran WebSockets.

Backend: PythonAnywhere. Plataforma específica para proyectos Python y Django que permite desplegar en minutos mediante cargar los archivos directamente desde el navegador. Su plan gratuito no soporta WebSockets ni dominios personalizados, lo que lo limita a prototipos tempranos o aplicaciones que no requieran comunicación en tiempo real. Adecuado para pruebas rápidas, no para producción.

Backend: AWS (Elastic Beanstalk + RDS). Amazon Web Services ofrece Elastic Beanstalk como servicio PaaS para desplegar Django y RDS como servicio de base de datos administrado. Ofrece free tier de 12 meses, pero requiere más configuración técnica que Render o Railway: entender los conceptos de regiones, instancias, security groups, load balancers y configuración de RDS. Adecuado cuando el proyecto crece y necesita la infraestructura robusta de AWS, pero tiene una curva de aprendizaje más alta.

Backend: Google Cloud Run. Permite ejecutar contenedores Docker de Django en un modelo serverless con escalado automático según la carga, con PostgreSQL administrado vía Cloud SQL. Es económico para proyectos con tráfico variable: no se paga cuando no hay tráfico, y se escala automáticamente cuando llega. Requiere conocer conceptos de contenedores y tener una cuenta de Google Cloud configurada con tarjeta de crédito, aunque Google ofrece créditos gratuitos para estudiantes.

Backend: Azure App Service. Servicio PaaS de Microsoft que soporta Python y Django con despliegue desde GitHub. Ofrece créditos gratuitos para estudiantes a través de Azure for Students, lo que lo hace accesible sin costo durante el periodo académico. Sin embargo, es menos común en el ecosistema Django que AWS o Google Cloud, lo que significa menos ejemplos, tutoriales y soporte comunitario específico para este stack.

Frontend: Vercel. Es la plataforma líder para desplegar aplicaciones React en 2026. Ofrece 6.000 minutos de build gratuitos al mes, despliegue automático desde GitHub, previews por pull request y una red global de más de 100 edge locations que reduce la latencia de carga a aproximadamente 70ms. Su limitación principal es que el free tier está restringido a proyectos no comerciales. Ideal para el frontend de VocalCode por rendimiento, facilidad de uso y costo cero durante el MVP.

Frontend: Netlify. Alternativa a Vercel que permite uso comercial en su free tier, incluye formularios nativos y autenticación básica. Ofrece solo 300 minutos de build al mes (20 veces menos que Vercel) y un rendimiento ligeramente inferior (TTFB de aproximadamente 90ms frente a 70ms de Vercel). Adecuada si se necesita uso comercial sin pagar y el volumen de build es bajo.

Para VocalCode, la combinación óptima es Render para el backend (Django + PostgreSQL + WebSockets) y Vercel para el frontend (React SPA), ambos con free tier suficiente para cubrir el MVP académico sin costo inicial.

Fuentes consultadas en este bloque

- *Docker Documentation* — Guía Oficial de Contenedores y Docker Compose — <https://docs.docker.com/>
- *Render Documentation* — Guía de Despliegue de Servicios Web y PostgreSQL — <https://docs.render.com/>
- *Railway Documentation* — Guía de Despliegue y Gestión de Infraestructura — <https://docs.railway.com/>
- *Vercel Documentation* — Plataforma de Despliegue para Aplicaciones Frontend y JAMstack — <https://vercel.com/docs>
- *Netlify Documentation* — Arquitectura de Despliegue y Edge Networks — <https://docs.netlify.com/>

- *Amazon Web Services — Conceptos de Computación e Infraestructura Cloud* — <https://aws.amazon.com/what-is-aws/>
- *Google Cloud Run Documentation — Ejecución de Contenedores Serverless* — <https://cloud.google.com/run/docs>
- *Microsoft Azure App Service Documentation — Documentación de Azure App Service* — <https://learn.microsoft.com/es-es/azure/app-service/>
- *PythonAnywhere Help Center — Configuración y Hosting para Proyectos Web Python* — <https://help.pythonanywhere.com/>

BLOQUE 11. MERCADO LABORAL

Conocer el mercado laboral de un stack tecnológico implica ir más allá de contar vacantes: significa entender qué empresas lo usan, qué perfiles profesionales buscan, qué salarios ofrecen, en qué sectores se aplica y hacia dónde evoluciona. Analizar el stack Django + React desde ambas perspectivas, la local y la internacional, permite tener una visión realista de las oportunidades profesionales que ofrece esta combinación y tomar decisiones informadas sobre la carrera profesional.

Colombia

Demanda. El Ministerio de Tecnologías de la Información y las Comunicaciones MinTIC, en conjunto con Fedesoft, identificó en su estudio de diciembre de 2025 una brecha de 85.000 vacantes digitales sin cubrir en 2026, con una demanda proyectada de 498.000 profesionales tecnológicos para los próximos cinco años. Esta brecha no se limita al sector tecnológico sino que atraviesa salud, educación, agroindustria, logística, manufactura y servicios financieros. Python se ha consolidado como uno de los lenguajes con mayor demanda para cubrir esta brecha.

Vacantes. No existe un registro público y oficial del número exacto de vacantes exclusivas para Django + React en Colombia. Las plataformas de empleo muestran miles de vacantes de Python, React y Django activas, pero incluyen todos los perfiles relacionados. La demanda es sumamente activa y existen vacantes reales que las empresas no logran cubrir por falta de personal capacitado.

Empresas y sectores. Django y React se utilizan en Colombia en empresas de desarrollo a la medida, fintechs como Prometeo, Addi y Nequi, edtechs, consultoras de nearshoring y entidades financieras como Bancolombia. Los sectores con mayor demanda son el financiero, educación, salud digital, e-commerce y gobierno digital, los cuales priorizan la seguridad y el cumplimiento normativo.

Perfiles profesionales. Los perfiles más solicitados son: desarrollador backend Django, desarrollador frontend React, desarrollador full stack Django + React, e ingeniero DevOps con experiencia en Python. La demanda de full stack es particularmente alta para agilizar el desarrollo.

Salarios aproximados. Según Computrabajo, un Desarrollador de software gana en promedio entre 3.291.842 y 3.759.224 pesos mensuales. Para perfiles especializados en Python, Talently ubica el salario junior alrededor de 1.913 USD mensuales, alcanzando hasta 6.000 USD en roles senior. Estas cifras integran el mercado local y el trabajo remoto para empresas extranjeras.

Internacional

Países con mayor demanda. Estados Unidos lidera la demanda de desarrolladores Django a nivel global, ofreciendo abundantes oportunidades senior y los salarios más altos. India es el segundo mercado más amplio. El Reino Unido mantiene una demanda sostenida para proyectos financieros, mientras que Canadá y Australia son destinos frecuentes para desarrolladores remotos.

Empresas. A nivel internacional, Django es utilizado por empresas como Instagram, Pinterest, Spotify, Dropbox, Mozilla y Zapier. Esta presencia demuestra que Django es un framework robusto orientado a entornos de producción empresarial de alto tráfico.

Trabajo remoto. El trabajo remoto es una modalidad consolidada para este stack. Existen portales como Arc.dev y Turing dedicados a vacantes remotas en Django. Las empresas destacan el crecimiento del nearshoring, donde compañías de Estados Unidos prefieren contratar en Latinoamérica por la coincidencia de horarios.

Tendencias. La encuesta de Stack Overflow destaca que PostgreSQL superó el 55% de uso y Python continuó su tendencia al alza. El crecimiento de Python en inteligencia artificial posiciona a Django como el framework de referencia para integrar modelos de IA en aplicaciones web.

Fuentes consultadas en este bloque

- Ministerio TIC Colombia y Fedesoft — Estudio de Empleabilidad y Talento Digital en Colombia — <https://www.mintic.gov.co/portal/715/w3-article-425772.html>
- Computrabajo — Salario de Programador/a en Colombia — <https://co.computrabajo.com/salarios/programadora>
- Computrabajo — Salario de Desarrollador/a de Software en Colombia — <https://co.computrabajo.com/salarios/desarrolladores-de-software>
- Talently — Estudios de Salarios Tech en LATAM — <https://talently.tech/>
- Stack Overflow — 2025 Developer Survey — <https://survey.stackoverflow.co/2025/>
- Arc.dev — Ofertas Laborales Remotas para Desarrolladores Django — <https://arc.dev/remote-jobs/django>
- Turing — Empleo Remoto para Ingenieros Python / Django — <https://www.turing.com/jobs/remote-python-django-jobs>
- ITJobsWatch UK — Métricas del Mercado Laboral para Django — <https://www.itjobswatch.co.uk/find/Python-Django-Developer-jobs>
- WeAreDevelopers — Vacantes Internacionales para Desarrolladores Django — <https://www.wearedevelopers.com/en/jobs/s/django>

BLOQUE 12. COMPARACIÓN TECNOLÓGICA

Para determinar cuándo es adecuado elegir el stack Django + React y cuándo conviene explorar alternativas, se compara a continuación con tres stacks ampliamente utilizados en el desarrollo web profesional: MERN (MongoDB, Express, React, Node.js), .NET Full Stack (ASP.NET Core + React/Blazor) y Spring Boot + React (Java). El análisis cubre diez criterios técnicos y de mercado que permiten tomar una decisión fundamentada.

Matriz comparativa

Criterio	Django + React	MERN (Node/Express/Mongo + React)	.NET Full Stack (ASP.NET Core)	Spring Boot + React (Java)
Rendimiento	Bueno para la mayoría de apps de gestión; más lento en cómputo puro por ser interpretado (Python).	Destaca en operaciones de E/S concurrente gracias a su modelo asíncrono no bloqueante (Node.js).	Rendimiento crudo muy alto en cómputo intensivo, al correr sobre el CLR optimizado.	Rendimiento crudo muy alto en cómputo intensivo, al correr sobre la JVM optimizada.
Escalabilidad	Escala adecuadamente para apps empresariales/educativas, apoyado en PostgreSQL y Celery.	Escala bien horizontalmente; arquitectura desacoplada como los demás stacks.	Ventaja histórica en sistemas de muy gran escala (banca, telecomunicaciones).	Ventaja histórica en sistemas de muy gran escala (banca, telecomunicaciones).
Seguridad	Ventaja de partida: protecciones activadas por defecto (CSRF, sanitización, cabeceras seguras).	Depende en mayor medida de la configuración del equipo, ya que Express es minimalista.	Seguridad robusta, pero con más configuración manual que Django.	Seguridad robusta, pero con más configuración manual que Django.
Productividad	Muy alta gracias a su filosofía de "baterías incluidas" (ORM, admin, formularios).	Alta velocidad de desarrollo inicial gracias a un único lenguaje de extremo a extremo (JavaScript).	Requiere más configuración inicial; compensa con herramientas maduras de tipado estático.	Requiere más configuración inicial; compensa con herramientas maduras de tipado estático.
Curva de aprendizaje	Una de las más accesibles gracias a la sintaxis clara de Python.	Exige dominar JavaScript en ambas capas, reduciendo el número de lenguajes a aprender.	Más pronunciada, por la verbosidad de C# y su ecosistema de configuración.	Más pronunciada, por la verbosidad de Java y su ecosistema de configuración.
Comunidad y documentación	Comunidad global grande y documentación oficial completa.	Comunidad global grande y documentación oficial completa.	Comunidad global grande y documentación oficial completa.	Comunidad global grande y documentación oficial completa.
Costos	Completamente gratuito y de código abierto.	Completamente gratuito y de código abierto.		Completamente gratuito y de código abierto.

Gratuito desde .NET Core, aunque su ecosistema tradicional (Windows Server, SQL Server) puede implicar licencias.

Mercado laboral	Demanda sólida y creciente, especialmente en proyectos que combinan desarrollo web con ciencia de datos / IA.	Volumen de vacantes ligeramente mayor a nivel global por su fuerte adopción empresarial.	Fuerte demanda en entornos corporativos con ecosistema Microsoft ya definido.	Volumen de vacantes ligeramente mayor a nivel global por su fuerte adopción histórica.
Cloud	Totalmente compatible con AWS, Azure, Google Cloud, Render, Railway o Heroku.	Totalmente compatible con AWS, Azure, Google Cloud, Render, Railway o Heroku.	Totalmente compatible con AWS, Azure, Google Cloud, Render, Railway o Heroku.	Totalmente compatible con AWS, Azure, Google Cloud, Render, Railway o Heroku.
Ecosistema IA / Data	Ventaja significativa: Python es el líder en IA, permitiendo integrar modelos directamente en el backend.	Limitado: requiere conectar Node.js con servicios externos de IA (Python) mediante APIs.	Creciente gracias a ML.NET, pero aún lejos del ecosistema maduro de Python.	Creciente con Spark y librerías JVM, pero aún lejos del ecosistema maduro de Python.

¿Cuándo elegir Django + React y cuándo elegir otra alternativa?

Elegir Django + React cuando: el proyecto requiere seguridad reforzada desde el inicio sin configuración extra, se necesita un desarrollo rápido (gracias a su panel de administración y ORM), y especialmente si se planea integrar capacidades de Inteligencia Artificial, procesamiento de datos o machine learning en el servidor, ya que Python es el lenguaje indiscutido en ese campo. Es ideal para plataformas educativas, fintechs, e-commerce robusto y SaaS empresariales de mediana escala.

Elegir MERN cuando: el equipo prefiere trabajar con un único lenguaje (JavaScript) de principio a fin, reduciendo la fricción entre frontend y backend. Es una excelente opción para startups, prototipos rápidos o aplicaciones donde el manejo intensivo de entrada/salida (I/O) en tiempo real (chats, notificaciones) es la prioridad principal sobre la seguridad estricta o el análisis de datos.

Elegir .NET o Spring Boot cuando: el proyecto es de muy gran escala empresarial (banca, telecomunicaciones, seguros), se cuenta con equipos grandes de desarrollo, y ya existe una infraestructura corporativa atada a Microsoft (.NET) o al ecosistema Java (Spring Boot). Estos stacks brillan por su rendimiento crudo en cómputo intensivo y sus estrictas tipificaciones estáticas, aunque sacrifican la velocidad de desarrollo inicial y la conexión nativa con el mundo de la IA.

Fuentes consultadas en este bloque

- Django Software Foundation. (2025). **The web framework for perfectionists with deadlines (Documentación oficial)**. <https://docs.djangoproject.com/>
- OpenJS Foundation. (2025). **Express - Fast, unopinionated, minimalist web framework for Node.js**. <https://expressjs.com/>

- Microsoft. (2025). **ASP.NET Core documentation**. <https://learn.microsoft.com/es-es/aspnet/core/>
- VMware. (2025). **Spring Boot Reference Documentation**. <https://docs.spring.io/spring-boot/reference/>
- MongoDB Inc. (2025). **MongoDB Documentation**. <https://www.mongodb.com/docs/>
- Stack Overflow. (2025). **Developer Survey Results 2025**. <https://survey.stackoverflow.co/2025/>

BLOQUE 13. PROPUESTA DE PROYECTOS

Con base en todo lo investigado a lo largo de esta guía, el equipo propone tres proyectos innovadores que aprovechan las fortalezas específicas del stack Django + React: su seguridad reforzada, la alta productividad del backend, la capacidad de manejo de estado del frontend y la posibilidad de integrar componentes de análisis de datos o inteligencia artificial.

Proyecto 1: VocalCode — Editor de código controlado por voz e IA

Problema. Las personas con discapacidad motriz en manos (por ejemplo, lesión medular, ELA, parálisis cerebral) prácticamente no tienen acceso a aprender o ejercer programación, porque los entornos de desarrollo (IDEs) están diseñados exclusivamente para teclado y mouse.

Justificación. Existen tecnologías de reconocimiento de voz, pero ninguna está diseñada para programar. Si un desarrollador dice "paréntesis", la tecnología escribe la palabra en lugar del símbolo "(". Además, programar requiere contexto: no basta con transcribir palabras, se necesita un sistema que entienda la intención del desarrollador y le asista en la escritura de código real.

Usuarios. Personas con discapacidad motriz que quieran aprender o trabajar en programación (como es el caso de uno de los integrantes de este equipo); instructores de inclusión tecnológica y programas educativos especializados.

Innovación. VocalCode combina dos tecnologías clave: (1) Un "diccionario inteligente de voz" que distingue entre comandos del teclado y nombres de variables. Si el usuario dice "función suma abrir paréntesis a coma b cerrar paréntesis", el sistema entiende que "abrir paréntesis" y "coma" son acciones del teclado, mientras que "suma", "a" y "b" son texto, generando: `function suma(a, b) {`. (2) Integración de Inteligencia Artificial que analiza el contexto de lo que se está programando para sugerir autocompletados, corregir errores de sintaxis dictados por voz y predecir la siguiente línea de código, reduciendo drásticamente la cantidad de comandos de voz necesarios.

Módulos. Módulo de reconocimiento de voz con diccionario técnico diferenciado (comandos vs. texto). Módulo de conexión a IA para autocompletado contextual y corrección de errores. Editor de código en el navegador con resaltado de sintaxis (basado en CodeMirror 6). Panel de historial de comandos de voz con posibilidad de editar el último comando dictado. Panel de progreso y ejercicios guiados completamente por voz.

Tecnologías. Frontend en React con Web Speech API (reconocimiento de voz nativo del navegador) y CodeMirror 6 para el editor. Backend en Django + DRF para guardar perfiles, diccionarios personalizados y progreso. Django actuará como puente para consumir una API de IA (como OpenAI API o un modelo local de LLM) que procese el contexto del código y devuelva sugerencias. WebSockets con Django Channels para que la interfaz reaccione en tiempo real a los comandos de voz y a las respuestas de la IA sin recargar.

Patrones posibles. Strategy (para intercambiar el intérprete de comandos según el lenguaje, ej. Python o JS). Proxy (Django actuando como intermediario seguro ante la API de IA, ocultando las claves de acceso). Observer (la interfaz del editor se actualiza automáticamente cuando llega una sugerencia de la IA).

Base de datos. PostgreSQL, para guardar perfiles de accesibilidad, configuraciones del diccionario de voz, historial de comandos usados y progreso en los ejercicios.

Justificación del uso del stack. Este proyecto justifica perfectamente la elección de Django + React. Por un lado, React es el único lado que puede acceder al micrófono del usuario de forma nativa y manejar el estado dinámico de un editor de código complejo. Por otro lado, Django (Python) es el lenguaje líder a nivel mundial en Inteligencia Artificial, lo que permite integrar modelos de lenguaje (LLMs) o servicios de IA de forma nativa, rápida y segura desde el backend, algo que sería mucho más complejo y limitado con Node.js o Java.

Proyecto 2: DataPulse — Plataforma de análisis masivo de datos y métricas de mercado

Problema. Comerciantes, analistas y estudiantes tienen acceso a grandes volúmenes de datos crudos en formatos como CSV o Excel (por ejemplo, catálogos de Amazon, ventas mensuales), pero carecen de conocimientos técnicos en Python o herramientas costosas como Tableau para limpiar esos datos, cruzarlos y extraer estadísticas reales que les ayuden a tomar decisiones de negocio.

Justificación. Los datos sin procesar no sirven para nada. Una plataforma web donde el usuario solo tenga que arrastrar un archivo masivo y recibir automáticamente gráficos de distribución, medias, outliers y tendencias, democratiza el análisis de datos sin necesidad de instalar software complejo en la computadora.

Usuarios. Vendedores en plataformas de e-commerce (Amazon, MercadoLibre), pequeños negocios que manejan inventarios en Excel, analistas de marketing y estudiantes que necesitan procesar datasets para sus trabajos.

Innovación. Procesar archivos con millones de filas de forma asíncrona en el servidor usando Python, para que el navegador del usuario no se congele. El sistema detecta automáticamente el tipo de dato, limpia errores comunes y genera un panel interactivo de estadísticas listo para descargar o presentar.

Módulos. Módulo de carga masiva de archivos soportando CSV y Excel pesados. Motor de limpieza y transformación de datos en el backend. Motor estadístico que calcula promedios, medianas, correlaciones y datos atípicos. Panel de visualización de datos interactivo con gráficos dinámicos. Módulo de exportación de los datos limpios y los gráficos a PDF o Excel nuevo.

Tecnologías. Backend Django + DRF complementado con Pandas (la librería estrella de Python para datos) y Celery para procesar los archivos pesados en segundo plano sin bloquear la app. Frontend en React con librerías de gráficos como Recharts o Chart.js para mostrar los resultados de forma interactiva.

Patrones posibles. Repository (para abstraer las operaciones complejas de Pandas y que el controlador no se llene de código matemático). Command (para ejecutar distintas tareas de análisis como "calcular media" o "encontrar errores" desde una cola). Observer (React se actualiza en tiempo real mediante WebSockets cuando Celery termina de procesar un archivo gigante).

Base de datos. PostgreSQL para guardar los perfiles de usuario, el historial de archivos subidos y las configuraciones de los gráficos. Los datos masivos procesados se guardan temporalmente en el servidor como archivos estáticos para no sobrecargar la base de datos relacional.

Justificación del uso del stack. Este proyecto justifica la elección de Django por encima de cualquier otro stack: Python es el lenguaje líder mundial en ciencia de datos gracias a Pandas. Hacer este análisis en Node.js (MERN) sería extremadamente lento y limitado. React, por su parte, es el mejor entorno para renderizar gráficos interactivos que respondan al movimiento del ratón sin recargar la página.

Proyecto 3: ExcelForge — Herramienta avanzada de fusión, comparación y transformación de Excel

Problema. Profesionales en administración, finanzas y logística pierden horas o días comparando versiones de archivos Excel masivos, uniendo base de datos o intentando actualizar un archivo maestro usando la información de otro archivo. Excel colapsa con archivos muy pesados y hacer esto a mano genera muchos errores humanos.

Justificación. Las fórmulas de Excel son frágiles y las macros son inseguras. Se necesita una herramienta web segura donde el usuario suba dos archivos, seleccione qué columnas coinciden entre ellos, y el servidor haga el cruce o la unión en segundos, devolviendo un archivo limpio y listo para usar.

Usuarios. Contadores, analistas financieros, recursos humanos que manejan nóminas enormes, y cualquier persona que trabaje diariamente manipulando hojas de cálculo corporativas.

Innovación. Un "diff visual" para Excel. En lugar de mostrar celdas feas con errores, la interfaz de React permite mapear columnas arrastrando y soltando (ej. "La columna A de este archivo es la misma que la columna C del otro"). El sistema resalta en colores qué filas faltan, cuáles sobran y cuáles se modificaron, permitiendo descargar el resultado fusionado.

Módulos. Módulo de carga múltiple de archivos Excel o CSV. Interfaz de mapeo visual de columnas para indicar al sistema cómo relacionar los datos. Motor de comparación que detecta diferencias exactas entre archivos. Motor de fusión que une los archivos basándose en las columnas mapeadas. Visor de diferencias con colores y descarga del archivo resultante.

Tecnologías. Backend Django + DRF utilizando OpenPyXL y Pandas, que son las librerías más potentes de Python para leer y escribir archivos de Excel pesados sin romperse. Frontend en React con una cuadrícula de datos interactiva para que el usuario pueda previsualizar el resultado antes de descargarlo.

Patrones posibles. Strategy (para intercambiar la lógica si el usuario elige "Unir archivos", "Comparar archivos" o "Actualizar un archivo con otro"). Adapter (para leer distintos formatos como .xlsx antiguo, .csv o .xls y que el sistema los trate a todos por igual). Facade (para ocultar toda la complejidad de Pandas detrás de un solo botón que dice "Procesar").

Base de datos. PostgreSQL para guardar las configuraciones de las uniones que el usuario quiera guardar para el futuro, y un registro de los archivos procesados. Al igual que en DataPulse, los archivos pesados se procesan en la memoria RAM del servidor y se entregan como descarga, no se guardan celda por celda en la base de datos.

Justificación del uso del stack. Al igual que en el Proyecto 2, la clave aquí es Python. Manipular archivos de Excel complejos, con miles de filas y distintos formatos, es el terreno natural de Python. Intentar leer un archivo Excel binario con JavaScript en el backend de Node.js es un dolor de cabeza. React permite crear la interfaz de arrastrar y soltar columnas de forma fluida, algo que sería muy torpe en Django templates tradicional.

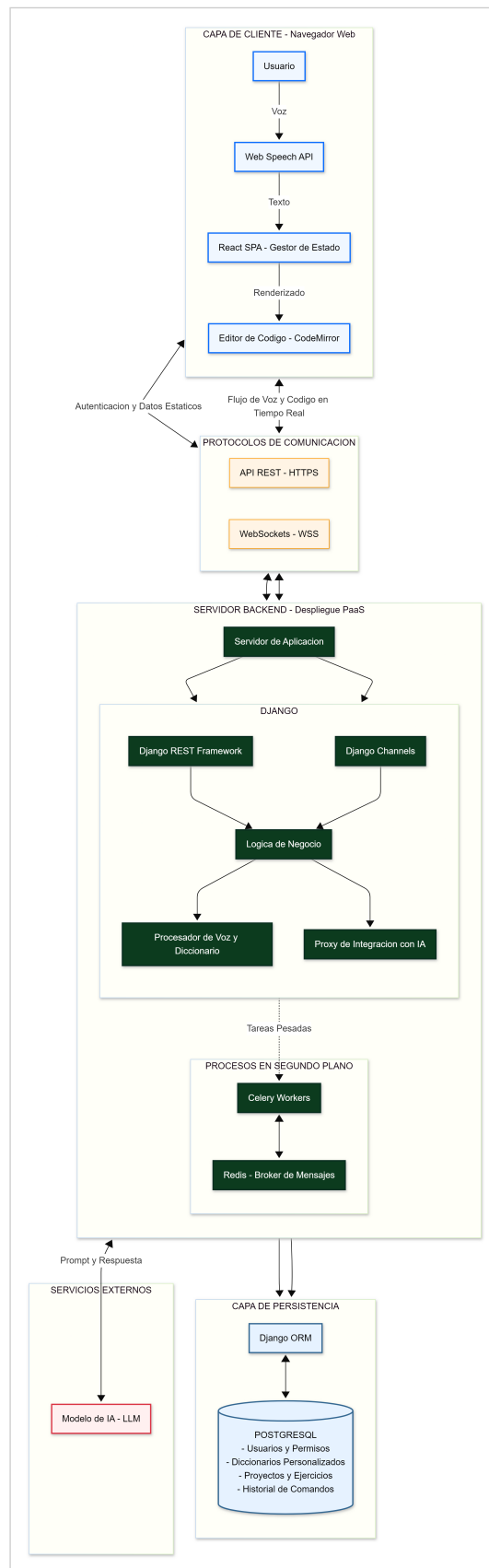
Recomendación final del equipo

Como grupo, recomendamos desarrollar en las siguientes guías el **Proyecto 1 (VocalCode)**, porque su alcance es el más ajustado al tiempo disponible del proceso formativo, resuelve una necesidad real de inclusión que nos motiva como futuros profesionales, y nos permite aplicar de forma clara conceptos clave investigados en esta guía (autenticación por JWT, manejo de estado complejo en React, comunicación en tiempo real con WebSockets mediante Django Channels y consumo de APIs del navegador), sin depender de integraciones geoespaciales complejas (como exige EcoRuta) o algoritmos de adaptación avanzados (como exige AulaViva).

Fuentes consultadas en este bloque

- Mozilla Developer Network. (2025). **Web Speech API**. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- CodeMirror. (2025). **CodeMirror 6 Documentation**. <https://codemirror.net/>
- PostGIS. (2025). **PostGIS — Spatial and Geographic objects for PostgreSQL**. <https://postgis.net/>
- Leaflet. (2025). **An open-source JavaScript library for mobile-friendly interactive maps**. <https://leafletjs.com/>
- Django Software Foundation. (2025). **Django Channels Documentation**. <https://channels.readthedocs.io/>

DIAGRAMA DE ARQUITECTURA DEL PROYECTO



Descripción de las capas

Capa de Cliente. Es todo lo que corre en el navegador de la computadora del usuario. Aquí es donde entra React como la interfaz visual, el editor de código CodeMirror donde se escribe, y la Web Speech API que es la tecnología nativa del navegador que escucha el micrófono y convierte la voz en texto.

Capa de Protocolos. Son los caminos por donde viaja la información entre el usuario y el servidor. Usamos dos: API REST con HTTPS para cosas normales como iniciar sesión o guardar proyectos, y WebSockets para la comunicación en tiempo real, lo cual es vital para que la voz y el código de la IA aparezcan al instante sin que la página tenga que recargarse.

Capa de Servidor Backend. Es el cerebro de la aplicación que corre en la nube. Lo manejamos con Django dividido en dos partes: Django REST Framework que atiende los pedidos normales, y Django Channels que maneja la conexión viva de los WebSockets. También está nuestra lógica de negocio, el procesador que entiende los comandos de voz y el Proxy de IA que se encarga de hablar con la inteligencia artificial de forma segura sin que las contraseñas queden expuestas.

Capa de Persistencia. Es donde guardamos toda la información de forma segura. El ORM de Django actúa como traductor para no tener que escribir bases de datos a mano, y se conecta a PostgreSQL, que es donde quedan guardados los usuarios, sus diccionarios de voz personalizados y el historial de lo que han programado.

Servicios Externos y Segundo Plano. Están fuera de nuestro servidor pero son clave. El modelo de IA es el que le da inteligencia al dictado para sugerir código. Además, tenemos a Celery y Redis que se encargan de hacer tareas pesadas en segundo plano para que el usuario no se quede esperando.

Explicación del flujo en el diagrama

El diagrama muestra claramente los dos caminos principales que sigue la información en VocalCode. El primer camino es el flujo estándar: cuando el usuario quiere ver sus proyectos guardados, React envía una petición por REST, llega a DRF, este le pregunta a la lógica de negocio, la lógica usa el ORM para buscarlo en PostgreSQL, y devuelve el dato por el mismo camino.

El segundo camino es el flujo en tiempo real, que es la innovación del proyecto: el usuario habla al micrófono, la Web Speech API captura la voz, React la manda inmediatamente por el WebSocket hasta Django Channels. Desde ahí, la lógica de negocio decide si usa el diccionario rápido o si le pregunta a la IA a través del Proxy. La IA devuelve el código, vuelve por el WebSocket y React lo pinta en el editor de CodeMirror. Todo esto pasa en milisegundos y sin recargar la página, logrando que una persona con discapacidad motriz pueda programar fluidamente solo con su voz.

Fuentes consultadas en este bloque

- Django Software Foundation — Django Channels documentation — <https://channels.readthedocs.io/>

- Mozilla Developer Network — Web Speech API — https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- CodeMirror — CodeMirror 6 documentation — <https://codemirror.net/>

BLOQUE . GLOSARIO DE TÉRMINOS

A continuación presentamos los términos técnicos clave que aprendimos durante la investigación, explicados con nuestras propias palabras y desde nuestra perspectiva como estudiantes.

Término	Definición
API	Para nosotros es el puente que comunica a React con Django, permitiéndoles pedir y enviar información sin que sus códigos se mezclen.
Backend	Es la parte oculta que corre en el servidor encargándose de la lógica pesada, en nuestro caso manejado por Django y PostgreSQL.
Biblioteca	La vemos como una caja de herramientas que usamos a nuestra manera, como React que solo pinta la pantalla sin darnos reglas estrictas.
CORS	Es un permiso de seguridad del navegador que le indica que está bien que nuestro frontend hable con el backend aunque estén en lugares distintos.
Cloud	Son servidores que alquilamos por internet para no tener que comprar y mantener máquinas físicas en casa para nuestra aplicación.
Django	Es el motor principal de nuestro servidor porque nos ahorra mucho tiempo dándonos herramientas ya listas para usar sin empezar de cero.
Docker	Es como una caja que empaqueta todo nuestro proyecto para que funcione exactamente igual en mi computadora que en la de mi compañero.
DRY	Es una regla de oro que significa no repetirnos, obligándonos a unir código duplicado en una sola función para no trabajar de más.
Framework	Es como el esqueleto de nuestra aplicación porque nos da reglas claras sobre cómo organizar los archivos desde el primer día.
Frontend	Es todo lo que el usuario final ve y toca en su pantalla, lo cual en nuestro proyecto se construyó completamente con React.
Git	Es el salvavidas que guarda cada cambio que hacemos en el código y nos deja trabajar en pareja sin el miedo de borrarlos el trabajo.
JWT	Es como un pase temporal que nos da Django al iniciar sesión para que React lo muestre y demuestre quién somos sin recargar la página.
ORM	Es nuestra salvación porque nos deja manejar la base de datos escribiendo Python en lugar de aprender y escribir consultas SQL a mano.

PaaS	Es la forma más fácil de subir nuestro proyecto a internet porque la plataforma se encarga del servidor y nosotros solo subimos el código.
Proxy	En VocalCode significa que usamos a Django como un mensajero seguro para hablar con la inteligencia artificial y esconder nuestras contraseñas.
Pull Request	Es la forma ordenada de pedirle a mi compañero que revise mi código antes de mezclarlo con el proyecto principal en GitHub.
React	Lo elegimos porque es la única manera de conectar el micrófono del usuario desde el navegador y hacer que la interfaz cambie al instante.
REST	Es la forma en que organizamos nuestra aplicación web asignándole una dirección única a cada dato y usando acciones normales para interactuar.
Runtime	Es el motor interno que hace funcionar el código, ya sea el intérprete de Python en el servidor o el motor del navegador en la pc del usuario.
Serverless	Es alquilar procesador por segundos en la nube, lo cual nos sirve para la página web pero no para Django porque este siempre debe estar despierto.
SOLID	Son cinco reglas de buen comportamiento que nos obligan a escribir código ordenado para que en el futuro no nos arrepintamos de haberlo hecho desordenado.
SPA	Es una página web inteligente que carga todo al principio y luego cambia la información sin hacer ruido de recarga, vital para que nuestra voz no se corte.
SQL	Es el idioma técnico que entiende la base de datos, pero gracias a nuestro framework nunca tuvimos que aprenderlo a fondo porque el programa lo habla por nosotros.
Strategy	Es una forma de programar tan inteligente que nos permite cambiar el cerebro de nuestra aplicación entre el diccionario y la inteligencia artificial sin romper nada.
WebSocket	Es como un túnel secreto y continuo entre el navegador y el servidor que usamos para ir mostrando el código de la inteligencia artificial letra por letra.
Web Speech API	Es la herramienta nativa del navegador que escucha lo que hablamos y lo convierte en texto, siendo la pieza fundamental de nuestro proyecto.
XSS	Es un tipo de ataque malicioso que intenta colar código extraño, pero React nos cuida de esto convirtiendo todo en texto normal de forma automática.

BLOQUE . REFERENTES BIBLIOGRÁFICOS

Listado consolidado de todas las fuentes consultadas por el equipo, organizadas por las categorías establecidas en la guía de aprendizaje.

Documentación oficial de los frameworks

- Django Software Foundation — Django FAQ — <https://docs.djangoproject.com/es/5.1/faq/general/>
- Django Software Foundation — Django design philosophies — <https://docs.djangoproject.com/en/6.0/misc/design-philosophies/>
- Django Software Foundation — Django authentication system — <https://docs.djangoproject.com/en/6.0/topics/auth/>
- Django Software Foundation — The Django admin site — <https://docs.djangoproject.com/en/6.0/ref/contrib/admin/>
- Django Software Foundation — Django release process — <https://docs.djangoproject.com/en/dev/internals/release-process/>
- Django Software Foundation — Migrations — <https://docs.djangoproject.com/en/6.0/topics/migrations/>
- Django Software Foundation — Async support — <https://docs.djangoproject.com/en/6.0/topics/async/>
- Django Software Foundation — About the Django Software Foundation — <https://www.djangoproject.com/foundation/>
- Django Software Foundation — Making queries — <https://docs.djangoproject.com/en/6.0/topics/db/queries/>
- Django Software Foundation — Class-based views — <https://docs.djangoproject.com/en/stable/topics/class-based-views/>
- Django Software Foundation — Signals — <https://docs.djangoproject.com/en/stable/topics/signals/>
- Django Software Foundation — Is Django an MVC framework? — <https://docs.djangoproject.com/en/6.0/faq/>
- Django Software Foundation — HTTP decorators — <https://docs.djangoproject.com/en/stable/topics/http/decorators/>
- Django Software Foundation — Database transactions — <https://docs.djangoproject.com/en/stable/topics/db/transactions/>
- Django Software Foundation — Custom management commands — <https://docs.djangoproject.com/en/stable/howto/custom-management-commands/>

- Django Software Foundation — Security in Django — <https://docs.djangoproject.com/en/5.1/topics/security/>
- Django Software Foundation — Password management in Django — <https://docs.djangoproject.com/en/5.1/topics/auth/passwords/>
- Django Software Foundation — Cross Site Request Forgery protection — <https://docs.djangoproject.com/en/5.1/ref/csrf/>
- Django Software Foundation — Models — <https://docs.djangoproject.com/en/5.1/topics/db/models/>
- Django Software Foundation — PostgreSQL specific functions — <https://docs.djangoproject.com/en/5.1/ref/contrib/postgres/>
- Django Software Foundation — Testing in Django — <https://docs.djangoproject.com/en/5.1/topics/testing/>
- Django Software Foundation — Django Channels documentation — <https://channels.readthedocs.io/>
- Django REST Framework — Django REST Framework documentation — <https://www.django-rest-framework.org/>
- Django REST Framework — Authentication — <https://www.django-rest-framework.org/api-guide/authentication/>
- Meta Platforms, Inc. — React documentation — <https://react.dev/>
- Meta Platforms, Inc. — React reference — <https://react.dev/reference/react>
- Meta Platforms, Inc. — React 19 — <https://react.dev/blog/2024/12/05/react-19>
- Meta Platforms, Inc. — Describing the UI — <https://react.dev/learn/describing-the-ui>
- Meta Platforms, Inc. — Responding to events — <https://react.dev/learn/responding-to-events>
- Meta Platforms, Inc. — dangerouslySetInnerHTML — <https://react.dev/reference/react-dom/components/common#dangerouslysetinnerhtml>

Documentación de lenguajes y herramientas del ecosistema

- Python Software Foundation — Functional Programming HOWTO — <https://docs.python.org/3/howto/functional.html>
- Python Packaging Authority — Python Packaging User Guide — <https://packaging.python.org/>
- Sébastien Eustace — Poetry documentation — <https://python-poetry.org/docs/>
- OpenJS Foundation — npm documentation — <https://docs.npmjs.com/>
- OpenJS Foundation — Yarn documentation — <https://yarnpkg.com/getting-started>
- OpenJS Foundation — Jest documentation — <https://jestjs.io/docs/getting-started>
- OpenJS Foundation — ESLint documentation — <https://eslint.org/>
- OpenJS Foundation — Express — <https://expressjs.com/>

- Astral — Ruff documentation — <https://docs.astral.sh/ruff/>
- Prettier — Prettier: Opinionated code formatter — <https://prettier.io/>
- pytest-django contributors — pytest-django documentation — <https://pytest-django.readthedocs.io/en/latest/>
- Callahan, K. — React Testing Library — <https://testing-library.com/docs/react-testing-library/intro/>
- Mehta, V. — Simple JWT — <https://django-rest-framework-simplejwt.readthedocs.io/en/latest/>
- Jones, C. — Introduction to JSON Web Tokens — <https://jwt.io/introduction>
- Internet Engineering Task Force — JSON Web Token RFC 7519 — <https://datatracker.ietf.org/doc/html/rfc7519>
- Celery Project — Celery documentation — <https://docs.celeryq.dev/en/stable/>
- Redis Ltd. — Redis documentation — <https://redis.io/docs/>
- Chainz, A. — django-cors-headers — <https://github.com/adamchainz/django-cors-headers>
- Microsoft — Playwright documentation — <https://playwright.dev/>
- Mozilla Developer Network — Web Speech API — https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- CodeMirror — CodeMirror 6 documentation — <https://codemirror.net/>
- Leaflet — An open-source JavaScript library for interactive maps — <https://leafletjs.com/>
- GitHub, Inc. — About pull requests — <https://docs.github.com/es/pull-requests>
- Chacon, S. — Pro Git — <https://git-scm.com/book/es/v2>
- Microsoft — Visual Studio Code documentation — <https://code.visualstudio.com/docs>
- JetBrains — PyCharm documentation — <https://www.jetbrains.com/pycharm/documentation/>

Documentación de bases de datos

- PostgreSQL Global Development Group — About PostgreSQL — <https://www.postgresql.org/about/>
- PostgreSQL Global Development Group — PostgreSQL documentation — <https://www.postgresql.org/docs/>
- PostGIS Project — PostGIS Spatial and Geographic objects — <https://postgis.net/>
- MongoDB Inc. — MongoDB documentation — <https://www.mongodb.com/docs/>

Documentación de plataformas Cloud

- Docker, Inc. — Docker documentation — <https://docs.docker.com/>
- Render, Inc. — Render documentation — <https://docs.render.com/>
- Railway Corp. — Railway documentation — <https://docs.railway.com/>
- Vercel, Inc. — Vercel documentation — <https://vercel.com/docs>

- Netlify, Inc. — Netlify documentation — <https://docs.netlify.com/>
- Amazon Web Services, Inc. — What is AWS? — <https://aws.amazon.com/what-is-aws/>
- Google Cloud — Cloud Run documentation — <https://cloud.google.com/run/docs>
- Microsoft — Azure App Service documentation — <https://learn.microsoft.com/es-es/azure/app-service/>
- Microsoft — API design — <https://learn.microsoft.com/es-es/azure/architecture/best-practices/api-design>
- PythonAnywhere LLP — Help center — <https://help.pythonanywhere.com/>

Libros técnicos y artículos académicos

- Chacon, S. — Pro Git — <https://git-scm.com/book/es/v2>
- Meijer, W., Trubiani, C., & Aleti, A. — Experimental evaluation of architectural software performance design patterns in microservices. Journal of Systems and Software — <https://doi.org/10.1016/j.jss.2024.112183>
- Martin, R. C. — The Clean Architecture — <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- Cockburn, A. — Hexagonal architecture — <https://alistair.cockburn.us/hexagonal-architecture/>
- Fowler, M. — Microservices — <https://martinfowler.com/articles/microservices.html>
- Fowler, M. — Repository pattern — <https://martinfowler.com/eaCatalog/repository.html>
- Fowler, M. — CQRS — <https://martinfowler.com/bliki/CQRS.html>
- Fowler, M. — YAGNI — <https://martinfowler.com/bliki/Yagni.html>
- Refactoring.Guru — Design patterns — <https://refactoring.guru/design-patterns>
- Refactoring.Guru — Principios SOLID — <https://refactoring.guru/es/solid>
- Python Software Foundation / ReactiveX — ReactiveX — <https://reactivex.io/>
- OWASP Foundation — OWASP Top 10 — <https://owasp.org/www-project-top-ten/>

Informes de mercado laboral

- Ministerio TIC Colombia y Fedesoft — Estudio de Empleabilidad y Talento Digital en Colombia — <https://www.mintic.gov.co/portal/715/w3-article-425772.html>
- Stack Overflow — 2025 Developer Survey — <https://survey.stackoverflow.co/2025/>
- Computrabajo — Salario de Programador/a en Colombia — <https://co.computrabajo.com/salarios/programadora>
- Computrabajo — Salario de Desarrollador/a de Software en Colombia — <https://co.computrabajo.com/salarios/desarrolladores-de-software>
- Talently — Estudios de salarios tech en LATAM — <https://talently.tech/>

Portales especializados en empleo tecnológico

- Arc.dev — Remote Django jobs — <https://arc.dev/remote-jobs/django>
- Turing — Remote Python Django developer jobs — <https://www.turing.com/jobs/remote-python-django-jobs>
- ITJobsWatch — Python/Django Developer trends — <https://www.itjobswatch.co.uk/find/Python-Django-Developer-jobs>
- WeAreDevelopers — Django jobs — <https://www.wearedevelopers.com/en/jobs/s/django>

Documentación de tecnologías comparadas

- Microsoft — ASP.NET Core documentation — <https://learn.microsoft.com/es-es/aspnet/core/>
- VMware — Spring Boot reference documentation — <https://docs.spring.io/spring-boot/reference/>

Enciclopedias y artículos divulgativos

- Wikipedia — Django framework — [https://es.wikipedia.org/wiki/Django_\(framework\)](https://es.wikipedia.org/wiki/Django_(framework))
- Wikipedia — Django Software Foundation — https://en.wikipedia.org/wiki/Django_Software_Foundation
- Wikipedia — Django web framework — [https://en.wikipedia.org/wiki/Django_\(web_framework\)](https://en.wikipedia.org/wiki/Django_(web_framework))
- SoloIngeniería — Django Framework en Python: Guía Completa — <https://www.soloingenieria.org/ingenieria-en-sistemas-computacionales/django-framework-en-python/>
- EcuRed — Django — <https://www.ecured.cu/Django>